

天津大学

本科生毕业论文



题目：处理器微架构侧信道漏洞安全测试技术研究

学 院	智能与计算学部
专 业	软件工程
年 级	2022 级
姓 名	杨宇鑫
学 号	3022207128
指导教师	鲁赵骏

独创性声明

本人声明：所呈交的毕业设计（论文），是本人在指导教师指导下，进行研究工作所取得的成果。除文中已经注明引用的内容外，本毕业设计（论文）中不包含任何他人已经发表或撰写过的研究成果。对本毕业设计（论文）所涉及的研究工作做出贡献的其他个人和集体，均已在论文中作了明确的说明。本毕业设计（论文）原创性声明的法律责任由本人承担。

论文作者签名：

杨宇鑫
2026年5月18日

本人声明：本毕业设计（论文）是本人指导学生完成的研究成果，已经审阅过论文的全部内容。

论文指导教师签名：

鲁超发
2026年5月18日

摘 要

云计算和多租户虚拟化的大范围部署，使微架构侧信道漏洞成为信息安全领域的重要威胁。高速缓存、分支预测、乱序执行、预取等硬件优化在提升计算速度的同时，亦留下了可被观测的时间差、功耗波动和资源竞争模式，攻击者无需越过权限边界即可从这些物理旁路中推断密钥、明文等受保护数据。本文以 AES 加密算法的 Flush+Reload 缓存侧信道攻击为研究对象，围绕微架构侧信道漏洞的安全测试技术展开研究。

论文工作分三个层次展开：理论层梳理了 Flush+Reload 攻击的底层机制，包括 CPU 缓存层次、共享内存与页面去重、高精度时间戳测量等支撑技术；设计实现层搭建了一套统一处理 AES-128/192/256 三种密钥长度的攻击框架，通过 Flush+Reload 配合排除法完成末轮子密钥恢复，对于 AES-192 和 AES-256，进一步借助加密 T 表的监控信息推导倒数第二轮子密钥，经密钥扩展逆运算反解出原始密钥；评估层通过对命中率、泄露带宽等统计量的测量来度量攻击效果。

实验在 Linux 平台上对随机噪声注入和缓存行刷新两种防御手段进行了对照测试，数据表明缓存刷新在阻断信息泄露方面最为彻底，能从根源上消除侧信道泄露通路。研究成果为微架构侧信道安全测试提供了可落地的方法论和工具支撑，可用于云平台风险评估与安全审计。

关键词： 微架构侧信道；Flush+Reload 攻击；AES 加密；缓存攻击；安全测试；信息泄露

ABSTRACT

With the proliferation of cloud computing and multi-tenant virtualization environments, processor microarchitectural side-channel vulnerabilities have become a major threat in the field of information security. These vulnerabilities stem from timing differences introduced by microarchitectural mechanisms—such as caches, branch prediction, and out-of-order execution that improve performance but can be observed, allowing an attacker to infer sensitive information without directly violating privilege boundaries. This thesis focuses on the Flush and Reload cache side-channel attack against the AES encryption algorithm and systematically investigates security testing techniques for microarchitectural side-channel vulnerabilities.

The thesis work proceeds at three levels: at the theoretical level, an in-depth analysis of the fundamental principles of the Flush+Reload attack is provided, covering key technical foundations including CPU cache structure, shared memory mechanisms, and timing measurement methods; at the design and implementation level, a unified attack framework supporting three key lengths—AES-128, AES-192, and AES-256—is constructed, which adopts Flush+Reload combined with an elimination method to recover the last round key, and for AES-192 and AES-256, further recovers the penultimate round key by reusing the encryption T-table monitoring data and then reconstructs the original key through the reverse key expansion process; at the evaluation level, a comprehensive quantitative metric system incorporating hit rate, leakage bandwidth, and attack success rate is established to assess attack effectiveness.

Experiments conducted under a Linux environment systematically evaluate two common defense measures—random noise injection and cache flushing—with results demonstrating that cache flushing is the most effective defense measure, eliminating the side-channel leakage path at its root. The research outcomes provide methodological and tool support for security testing of processor microarchitectural side-channel vulnerabilities, and offer reference value for cloud platform security assessment and security auditing.

KEY WORDS: Microarchitectural Side Channel; Flush+Reload Attack; AES Encryption; Cache Attack; Security Testing; Information Leakage

目 录

第一章	绪论	1
1.1	研究背景与意义	1
1.2	国内外研究现状	2
1.2.1	缓存侧信道攻击研究	2
1.2.2	AES 缓存攻击研究	5
1.2.3	微架构侧信道防御研究	6
1.2.4	瞬态执行攻击研究	7
1.2.5	现有研究不足与本文创新性	7
1.3	本文研究内容	8
1.4	论文组织结构	9
第二章	相关技术背景	10
2.1	CPU 缓存结构	10
2.1.1	缓存层次结构	10
2.1.2	缓存行与映射方式	11
2.1.3	缓存替换策略	12
2.1.4	缓存寻址：虚拟索引与物理标签	12
2.1.5	包含性与非包含性缓存层次	13
2.1.6	缓存一致性协议	14
2.1.7	MESI 协议如何使跨核攻击成为可能	15
2.1.8	缓存访问时间差异	15
2.2	共享内存机制	15
2.2.1	共享内存概述	15
2.2.2	POSIX 共享内存 API	16
2.2.3	写时复制与代码段共享	16
2.2.4	虚拟化环境中的内存去重	17
2.3	时间测量方法	18
2.3.1	时间戳计数器	18
2.3.2	RDTSC 与 RDTSCP 的区别	18
2.3.3	用户态时间测量的精度问题	18
2.3.4	时间阈值确定	19
2.4	AES 加密算法原理	20
2.4.1	AES 算法概述	20
2.4.2	AES 数学基础	20
2.4.3	AES 状态矩阵	21
2.4.4	AES 轮函数	21

2.4.5	AES 一轮运算示例	23
2.4.6	AES 密钥扩展	24
2.4.7	AES T 表实现	25
2.4.8	T 表与缓存行的映射关系	26
2.5	Flush+Reload 攻击原理	27
2.5.1	攻击模型	27
2.5.2	攻击流程	27
2.5.3	Flush+Flush 变体	28
2.5.4	与其他缓存攻击方法的比较	28
2.6	缓解措施概述	29
2.6.1	软件层面缓解	29
2.6.2	硬件层面缓解	30
2.7	本章小结	31
第三章	攻击方法设计与实现	32
3.1	第一阶段：恢复最后一轮子密钥	32
3.1.1	AES 最后一轮特征	32
3.1.2	T 表结构与监控策略	32
3.1.3	最后一轮的 T 表访问方式与攻击前提	34
3.1.4	T 表查表索引与密钥的代数关系	34
3.1.5	排除法的数学推导	36
3.1.6	AES-128 密钥恢复	38
3.2	第二阶段：恢复倒数第二轮子密钥	38
3.2.1	复用监控信息与等效块解密	38
3.2.2	从密文到 S_{13} 的推导与缓存行映射	41
3.2.3	复用监控数据直接恢复 Kd_1	42
3.2.4	从 Kd_1 计算倒数第二轮子密钥	43
3.2.5	从轮密钥恢复原始密钥	43
3.2.6	两阶段攻击流程总结	44
3.3	系统架构设计	45
3.3.1	进程间通信协议	45
3.4	量化评估指标体系	46
3.4.1	基础指标	46
3.4.2	时间统计指标	46
3.4.3	泄露带宽	46
3.4.4	攻击成功率	48
3.5	缓解措施实现	48
3.5.1	CPU 核心绑定	49
3.5.2	随机噪声注入	49
3.5.3	缓存刷新	49
3.6	本章小结	50

第四章	实验与分析	51
4.1	实验环境	51
4.1.1	硬件平台	51
4.1.2	软件环境	52
4.1.3	实验参数	52
4.2	基准测试结果	52
4.2.1	攻击成功率	52
4.2.2	缓存时间分布与阈值校准	53
4.3	缓解措施对比测试	53
4.3.1	噪声缓解措施机制	53
4.3.2	AES-128 缓解措施对比	54
4.3.3	AES-192 缓解措施对比	54
4.3.4	AES-256 缓解措施对比	55
4.3.5	三种 AES 缓解措施对比	57
4.4	量化指标分析	58
4.4.1	泄露带宽分析	58
4.4.2	泄露带宽与攻击成功率的关系	58
4.5	本章小结	59
第五章	总结与展望	60
5.1	工作总结	60
5.2	未来工作展望	61
参考文献		62

第一章 绪论

1.1 研究背景与意义

处理器微架构侧信道漏洞，近十年来一直是安全研究的活跃领域。从学术界到工业界，大量工作围绕该问题展开。

现代 CPU 为了指令吞吐率，在硬件层面引入了多种优化机制。多级缓存填补处理器与主存之间的速度鸿沟，分支预测器提前预测跳转方向，乱序执行引擎使指令在操作数就绪后立即发射，硬件预取器根据访问模式提前将数据预取至缓存。但这些微架构组件在提升性能的同时，也在执行路径上留下了可被外部观测的物理痕迹——访存延迟的细微涨落、功耗曲线的局部起伏、共享资源仲裁的顺序差异。探测方利用这些旁路信号，无需直接攻破权限隔离机制，即可推断出被攻击进程内部的敏感信息。

云计算的大范围部署，使微架构侧信道攻击已构成现实威胁。公有云平台上，成千上万的租户共享同一批物理服务器。虚拟机之间虽然逻辑隔离，底层 CPU 缓存、分支预测器、内存控制器等硬件资源仍然是共享的。Ristenpart 等人早在 2009 年就证实了同一亚马逊 EC2 实例上并置虚拟机的可行性^[1]，为后续跨虚拟机旁路攻击研究奠定了基础。Zhang 等人进一步推进了该方向，在 PaaS 云环境中实现了跨租户的缓存侧信道攻击^[2]。他们证明了：即使没有管理员权限，监听方仅依靠缓存访问模式即可推断同平台其他用户的敏感信息。国内学者同样关注到这一威胁——张伟娟等人对缓存侧信道攻防技术进行了系统综述^[3]，王崇等人从防御角度梳理了缓存侧信道的研究进展^[4]。

其根本原因在于：多租户共享硬件，该部署模型本身就是微架构侧信道攻击的温床。

AES 是缓存旁路攻击的主要目标之一。就应用范围而言——TLS/SSL、磁盘加密、VPN 隧道、数据库加密，几乎所有需要数据保密性的场景均采用 AES^[5]。为了追求性能，AES 的软件实现普遍将 SubBytes、ShiftRows 和 MixColumns 三个操作合并为预计算查找表，即 T 表^[6]。查表时的内存访问模式直接受密钥值决定，缓存命中与未命中的时间差异即成为信息泄露的标志。再加上 AES 密钥扩展算法是确定且可逆的：探测方只要恢复最后一轮（和倒数第二轮）子密钥，即可逆推出原始密钥。

以 AES 的 T 表实现为例，每轮加密的核心运算均转化为查表：四张 T 表 Te_0

至 Te_3 各含 256 个 32 位入口，每张表 1024 字节，恰好占 16 个 64 字节的缓存行。查表索引由明文字节与轮密钥字节异或决定，索引的高 4 位决定访问哪个缓存行。探测方通过 Flush+Reload 监控 T 表缓存行的访问状态，样本累积足够多后，排除法可将每个密钥字节的 256 个候选逐步收敛至唯一值。

2018 年 Spectre^[7] 和 Meltdown^[8] 的公开发布标志着微架构安全研究的新阶段。Spectre 利用分支预测器的误预测诱导 CPU 进入瞬态执行窗口，在窗内访问受保护数据，再通过缓存侧信道将数据外泄。Meltdown 则利用乱序执行绕过特权检查，直接读取内核地址空间。这意味着微架构优化本身已成为安全威胁的根源。Spectre 和 Meltdown 的披露，使得缓存侧信道从一种独立的攻击路径，升级为更复杂的瞬态执行攻击链中的关键数据外泄环节——微架构侧信道（主要是缓存）是数据外泄的通道，而瞬态执行是通向该通道的门户。在此之后，Foreshadow^[9] 针对 Intel SGX 可信执行环境发起攻击。RIDL^[10] 和 ZombieLoad^[11] 分别利用处理器内部缓冲区和填充缓冲区泄露数据。Canella 等人对瞬态执行攻击进行了系统分类^[12]，从该分类中可见微架构侧信道漏洞的层次。

另一方面，微架构侧信道漏洞的影响已超出学术圈。Intel、AMD、ARM 等芯片厂商被迫进行微码更新、操作系统补丁、编译器缓解，每次均需性能和安全之间进行权衡，但并不能消除漏洞的影响。云服务商在多租户隔离方面必须考虑安全性，某些场景下甚至直接为安全敏感客户提供独占物理机。此外，软件供应链不断延伸，第三方库和开源组件广泛嵌入，平台需对侧信道暴露面进行持续度量和自动化巡检，对传统的渗透测试和安全审计流程进行微架构侧信道的漏洞排查。

本课题以微架构侧信道安全测试为主题，选取 AES 加密算法的 Flush+Reload 缓存侧信道攻击作为分析对象，设计并交付可运行的测试用例与评估报告，完成风险检测与效果度量，输出标准化的测试流程、实验数据，为配置加固、安全审计提供参考指标。

1.2 国内外研究现状

1.2.1 缓存侧信道攻击研究

侧信道攻击的思想可追溯到上世纪九十年代 Kocher 的工作。1996 年，Kocher 发现不同密钥值会使密码运算产生可测量的时间差，于是提出了计时攻击^[13]。他证明了：运行时间本身即可泄露密钥信息。1999 年，他又提出了差分功耗分析^[14]，利用统计手段从功耗曲线中提取密钥相关信息。此后几乎所有侧信道攻击研究均可追溯至这两篇论文的思路：发现新的旁路信号、建立模型，进而逐步提升攻

击效率。

2002 年, Page 从理论上分析了缓存能否作为密码分析的旁路^[15]。结论是肯定的——缓存命中与未命中之间的时间差, 可被用于推断密钥信息。2005 年, Bernstein 提出了利用缓存计时差异恢复 AES 密钥的完整方案^[16], 在真实系统上实现了远程缓存计时攻击。整个攻击无需物理接触目标机器, 仅通过网络测量加密操作的响应时间, 即可逐步缩小密钥搜索空间。同一时期, Percival 的工作^[17]从另一个方向印证了该威胁: 他针对 OpenSSL 的 RSA 实现, 展示了缓存冲突如何导致密钥泄露。

2006 年, Osvik、Shamir 和 Tromer 给出了两种缓存攻击策略: Evict and Time 和 Prime and Probe。Evict and Time 的思路是先填满缓存将目标数据逐出, 再触发受害方加密、测量时间。时间差异暴露了缓存访问模式。Prime and Probe 则相反: 先将目标缓存组填满自身数据 (Prime 阶段), 待受害方执行完毕后观察哪些位置被替换 (Probe 阶段), 从而推断受害方访问了哪些缓存组。Prime and Probe 无需共享内存, 适用范围更广, 但精度受到缓存组粒度的限制。Tromer 等人后来发表了更完整的分析^[18], 将缓存攻击对 AES 的威胁进行了量化, 即通过少量缓存访问观测即可恢复完整 AES 密钥。

Flush+Reload 方法是 Yarom 和 Falkner 在 2014 年完整发表的^[19]。高分辨率和跨核心能力使其迅速成为该领域的标志性工具。其核心方法是借助 x86 的 CLFLUSH 指令将指定缓存行从整个缓存层级中逐出, 然后测量重载时间——如果目标数据被受害方带入缓存, 再次访问的时间就短, 否则就长。与 Prime+Probe 相比, Flush+Reload 的分辨率达到缓存行级别 (通常 64 字节), 噪声也更低。其跨核心能力依赖于共享 L3 缓存和 MESI 一致性协议。Benger 等人将其应用于椭圆曲线密码^[20], 证明仅凭少量侧信道信息即可恢复完整密钥。

Liu 等人在 2015 年证明了 LLC 上 Prime+Probe 攻击的可行性^[21]。跨虚拟机场景下恢复 AES 密钥, 该成果对云安全评估产生了深远影响。Gruss 等人研究的 Cache Template Attacks^[22]将缓存攻击推向了自动化: 无需预先获知受害方的内存布局, 通过模板匹配即可自动发现和利用缓存的侧信道泄露。他们此后又提出了 Flush+Flush 攻击^[23], 以 CLFLUSH 指令本身的执行时间代替内存访问时间作为信号, 进一步降低了被检测的风险。

Shusterman 等人在 JavaScript 等脚本环境中也进行了验证^[24]: 浏览器沙箱内仍可实现缓存侧信道攻击。无需在目标机器上安装任何本地程序, 网页中的一段 JavaScript 代码即可监视同机器上其他进程的缓存访问模式。这显著降低了攻击门槛。Gruss 等人从另一个方向发力, 利用 JavaScript 实现了 Rowhammer 攻击^[25], 使浏览器环境对微架构安全的威胁引起了广泛关注。

将这十几年的演进串联来看，缓存侧信道攻击呈现出清晰的发展脉络，可划分为三个阶段。第一阶段为理论分析与概念验证（2002–2006 年）：Page 从理论上论证了缓存作为密码分析旁路的可行性，Bernstein 在真实系统上实现了远程缓存计时攻击，Percival 针对 RSA 实现验证了缓存冲突导致密钥泄露的风险，Osvik 等人则确立了 Evict+Time 和 Prime+Probe 两种基本攻击范式。这一阶段的核心贡献在于证明了缓存侧信道威胁的现实性，并为后续研究奠定了方法论基础。第二阶段为实用化与精度提升（2007–2014 年）：Flush+Reload 方法的提出解决了高分辨率和跨核心两个关键问题，使跨核心、跨虚拟机的攻击从理论构想变为工程现实；Gullasch 等人提出的消去法则大幅降低了对观测精度的要求，使噪声环境下的密钥恢复成为可能。该阶段的标志性特征是攻击从实验室环境走向真实云平台，Irazaqui 等人在跨虚拟机场景下成功恢复 AES 密钥即为典型例证。第三阶段为自动化与攻击面扩展（2015 年至今）：Cache Template Attacks 使攻击发现实现了自动化，无需预先获知受害方内存布局即可自动识别和利用侧信道泄露；Shusterman 等人在浏览器 JavaScript 环境中实现了缓存侧信道攻击，打破了攻击需在目标机器上运行本地代码的传统前提；更为重要的是，Spectre 和 Meltdown 的披露使缓存侧信道从独立的攻击手段升级为瞬态执行攻击链中不可或缺的数据外泄环节，Gruss 等人提出的 Flush+Flush 变体则进一步降低了攻击被检测的风险。总体而言，缓存侧信道攻击的发展趋势表现为：空间分辨率从缓存组级别提升至缓存行级别，攻击场景从本地扩展至跨核心、跨虚拟机乃至浏览器环境，攻击方法从手工构造演进为自动化模板匹配，侧信道角色从独立攻击路径演变为复杂攻击链的基础组件。

国内研究方面，学者们从多个角度对微架构侧信道安全进行了系统性的梳理与研究。吴晓慧等人对微架构瞬态执行攻击与防御方法进行了系统总结^[26]，梳理了 Spectre、Meltdown 及其变种的技术原理和缓解方案。杨帆等人从可信执行环境角度梳理了软件侧信道攻击的研究进展^[27]，分析了 SGX 等可信执行环境面临侧信道攻击的脆弱性。唐博文等人从处理器时间侧信道角度综述了攻防技术的最新进展^[28]，对时间侧信道的攻击模型和防御机制进行了分类比较。Ge 等人的综述^[29] 则从更宏观的视角对微架构计时攻击和防御措施进行了系统性整理，涵盖了从理论分析到工程实践的全链条。张伟娟等人对缓存侧信道攻防技术进行了系统综述，从攻击模型角度对现有攻击方法进行了分类比较。王崇等人从防御角度梳理了缓存侧信道的研究进展，对硬件级、操作系统级和软件级防御方案的效果和局限性进行了分析。国内研究的特点在于侧重系统性综述和防御技术梳理，在攻击方法的原创性实验和量化评估方面仍有深入空间。

1.2.2 AES 缓存攻击研究

AES 的 T 表实现是缓存旁路攻击的主要入口。标准 AES 的软件实现为提升性能，通常将 SubBytes、ShiftRows 和 MixColumns 三个操作合并为查表。每个输出字节由 T 表查表异或得到，查表索引直接受输入明文和轮密钥的影响。监听方通过观测 T 表访问模式，即可逐步缩小密钥候选空间。

Aciicmez 和 Koç 在 2006 年提出了基于追踪驱动的 AES 缓存攻击^[30]。方法是将单次加密过程中所有缓存命中和未命中事件收集起来，构建一条完整的缓存访问轨迹，从中获取密钥信息。此后研究发现分支预测器亦可作为侧信道^[31]，通过条件分支的执行路径推断 AES 内部状态。Neve 和 Seifert 对访问驱动的缓存攻击进行了改进^[32]，提升了效率。

Gullasch 等人在 2011 年将访问驱动缓存攻击推向实用^[33]。他们提出的消去法（Elimination Method）是 AES 缓存攻击领域的一个里程碑。核心思路是：探测方无需精确获知哪个缓存行被访问了，只需知道哪个缓存行未被访问。未命中的信息即足以将不可能的密钥候选排除。这一思路大幅降低了对观测精度的要求，在噪声较大的真实环境中仍可实现。本文的攻击框架亦将排除法作为核心方法之一。

跨虚拟机场景下的 AES 缓存攻击是云安全研究的重要课题。Irazoqui 等人在 2014 年实现了跨虚拟机的 AES 缓存攻击^[34]。他们利用 LLC 的共享特性，在一个虚拟机中监视另一个虚拟机中 AES 实现的缓存访问，数分钟即获取了完整密钥。2016 年他们又将攻击应用到了跨处理器场景^[35]——即使探测方和被攻击方运行于不同物理 CPU 上，只要共享一个 LLC，攻击亦可成功。这些工作对云服务商的安全策略具有现实威胁。

从另一维度审视侧信道缓存攻击，按信息利用方式大致可划分为三类。

计时驱动，以 Bernstein 的成果为代表。测量加密操作的总执行时间，据此推断缓存访问模式。优势在于实现简单、无需共享内存，局限在于精度低、噪声多。

访问驱动，Osvik 等人的 Prime+Probe 和 Gullasch 等人的消去法均属于此类。直接观测缓存行的访问状态获取信息，精度高，但前提是要么具备共享内存，要么对缓存布局有充分了解。

追踪驱动，Aciicmez 的工作是典型。将单次加密的完整缓存访问轨迹完整记录后再进行分析，信息量最大，代价是观测难度亦最高。

本文采用的 Flush+Reload 属于访问驱动这一分支，配合排除法进行高精度密钥恢复。

综观 AES 缓存攻击的研究进展，可以观察到以下演进趋势：在攻击精度方面，从 Bernstein 的计时驱动攻击（仅能测量加密总时间）到 Gullasch 等人的消

去法（利用缓存行级未命中信息），攻击的空间分辨率和信息利用率持续提升；在攻击场景方面，从单机本地攻击发展到 Irazoqui 等人的跨虚拟机攻击和跨处理器攻击，攻击的适用范围不断扩大；在密钥恢复策略方面，现有工作大多针对 AES-128 设计，对 AES-192 和 AES-256 的攻击方案相对匮乏，且鲜有研究探讨如何复用同一批监控数据实现多阶段密钥恢复。这些不足为本文的研究提供了切入点。

1.2.3 微架构侧信道防御研究

面对日益严峻的缓存侧信道威胁，学界和工业界从不同层面提出了应对方案。防御思路大致分布在硬件、操作系统和软件三个层级上。

硬件级防御的核心思想是修改缓存架构以消除侧信道泄露。Wang 和 Lee 在 2007 年提出了新型缓存设计方案^[36]，利用随机化缓存映射和分区来破坏监听方对缓存状态的观测能力。Kim 等人提出的 STEALTHMEM^[37] 在操作系统层面进行缓存分区，为安全敏感进程划分专属缓存区域，从根本上阻断了共享缓存带来的侧信道风险。Intel 此后推出的 Cache Allocation Technology (CAT) 亦采用类似路线，具备硬件分区能力，但粒度和灵活度均有限。Tromer 等人对 AES 缓存攻击的防御进行了系统分析，涵盖查找表预加载、缓存行对齐等工程手段。

操作系统级防御主要依赖调度和隔离。核心绑定将敏感进程锁定在特定 CPU 核心上，避免与其他进程共享 L1/L2 缓存。虚拟机亲和性调度尽量将安全等级不同的虚拟机放置到不同物理 CPU 上。但其代价亦很明显：降低了调度灵活性，且无法对 LLC 级别攻击进行防御。

软件级防御部署最为便捷，亦是本文实验中重点评估的方向。常见的方法包括：敏感操作完成后主动刷新缓存行（CLFLUSH 指令），确保缓存中不残留敏感数据痕迹；加密操作中插入随机延迟或噪声访存，干扰时间测量；常数时间实现，保证所有代码路径的执行时间和内存访问模式与密钥无关。常数时间实现从根本上解决问题，但需大量改造现有代码，且存在性能开销。

然而，各类防御方案在实际部署中各有局限性。硬件级方案需修改处理器设计，周期长、成本高，已部署的芯片无法修复。操作系统级方案依赖调度策略，资源利用率与安全性之间难以兼顾。软件级方案部署灵活，但效果参差不齐，这正是本研究的出发点：既关注攻击本身，亦关注防御措施的量化评估方法。

从防御研究的整体格局来看，现有工作存在以下不足：第一，大多数防御方案的效果评估停留在定性描述层面，缺乏统一的量化指标体系来客观度量防御措施对信息泄露的抑制程度；第二，针对 AES-192 和 AES-256 等较长密钥变体的攻击与防御评估较少，已有工作主要聚焦于 AES-128；第三，噪声注入等概率性

防御方案的有效性分析多基于理论推导，缺乏在真实攻击框架下的系统性实验验证。本文旨在填补上述空白，通过构建统一的攻击测试框架，建立包含泄露带宽在内的量化评估指标体系，对多种防御措施进行系统性的实验评估。

1.2.4 瞬态执行攻击研究

2018 年初, Spectre 和 Meltdown 公开发布, Spectre 利用分支预测器的误预测诱导瞬态执行, Meltdown 利用乱序执行绕过特权检查, 二者均将缓存侧信道作为数据外泄通道。此后 Foreshadow 针对 Intel SGX 可信执行环境发起攻击, 利用 L1 终端故障 (L1 Terminal Fault) 绕过 SGX 的内存隔离保护。RIDL 和 ZombieLoad 分别利用处理器内部缓冲区和填充缓冲区泄露数据, 表明即使不共享缓存行, 微架构内部缓冲区亦可成为侧信道泄露源。Brasser 等人^[38]在 SGX 环境中亦验证了缓存侧信道攻击的可行性。Canella 等人对瞬态执行攻击进行了统一分类, 从误预测类型、瞬态执行指令窗口和数据外泄方式三个维度建立了系统化的分类框架。

这些工作指向同一事实: 缓存侧信道是几乎所有瞬态执行攻击的数据外泄基础。在瞬态执行攻击时代, 缓存侧信道攻防研究仍然是最底层的基础问题。瞬态执行攻击的涌现也从侧面凸显了缓存侧信道安全测试的重要性: 只有充分理解缓存侧信道的信息泄露机理并建立有效的量化评估方法, 才能为瞬态执行攻击的防御提供底层支撑。

1.2.5 现有研究不足与本文创新性

综合上述国内外研究现状, 现有工作在以下方面存在不足, 本文针对这些不足提出了相应的创新性贡献。

第一, 针对 AES-192 和 AES-256 的缓存侧信道攻击方案较为匮乏。现有 Flush and Reload 攻击研究主要聚焦于 AES-128, 对 AES-192 和 AES-256 的攻击仅停留在理论可行性讨论层面, 缺乏完整的密钥恢复方案和实验验证。本文提出了基于等效块解密的两阶段密钥恢复方法: 第一阶段通过排除法恢复最后一轮子密钥, 第二阶段复用同一批 Flush+Reload 监控数据, 结合等效解密中间值的代数推导直接恢复等效解密轮密钥 Kd_1 , 经 MixColumns 变换得到加密倒数第二轮子密钥。该方法的核心创新在于利用 AES 等效解密结构将缓存访问信息转化为倒数第二轮密钥约束条件, 且两个阶段共享同一批监控数据, 无需重复采样。

第二, 现有防御措施的评估缺乏统一的量化指标体系。大多数研究对防御效果的描述停留在定性层面, 如“攻击成功率下降”或“信息泄露减少”, 缺乏客观的量化度量。本文建立了涵盖命中率、泄露带宽和攻击成功率的量化评估指标体

系，其中泄露带宽定义为单次观测互信息的 4 倍与采样频率的乘积，从信息泄露速率角度刻画侧信道威胁程度，为不同防御方案的横向比较提供了统一基准。

第三，现有攻击框架大多针对单一密钥长度设计，缺乏对多种 AES 变体的统一支持。本文构建了一套统一处理 AES-128/192/256 三种密钥长度的攻击测试框架，采用模块化设计，通过差异化的密钥恢复流程适配不同密钥长度，同时保持 T 表监控策略和排除法核心逻辑的一致性，为微架构侧信道安全测试提供了可复用的工具支撑。

1.3 本文研究内容

本课题以微架构侧信道安全测试为总目标，围绕 AES 加密算法的 Flush+Reload 缓存旁路攻击展开。研究工作划分为三个层次：理论、设计实现、评估。

理论层，研究 Flush+Reload 攻击的底层机制和数学基础。具体包括：CPU 缓存层次结构与一致性协议、共享内存与页面去重机制、高精度时间戳测量；AES 分组密码的算法细节与 T 表加速实现；排除法的数学构造——从单样本分析到多样本累积的统计收敛，以及期望样本量的理论推导；针对 AES-192 和 AES-256 的两阶段密钥恢复策略：如何复用同一批监控数据，结合解密代数推导恢复倒数第二轮子密钥。理论层的工作目标是建立完整的攻击数学模型，为后续的设计实现提供推导依据。

设计实现层，构建了一套能统一处理 AES-128/192/256 三种密钥长度的攻击测试框架。框架采用父子进程模型：父进程作为监听方运行 Flush+Reload 采样，子进程作为被攻击方运行 AES 加密，进程间通过管道同步通信。共享内存模块利用 POSIX 共享内存机制存放 T 表数据，以缓存行为单位监视 T 表访问。密钥恢复模块实现了排除法，通过累积未命中信息逐步排除错误候选——AES-128 恢复最后一轮子密钥后逆推原始密钥，AES-192 和 AES-256 采用两阶段恢复流程：阶段一恢复最后一轮子密钥，阶段二复用同一批监控数据，结合等效解密中间值直接恢复等效解密轮密钥 Kd_1 ，再经 MixColumns 变换得到加密倒数第二轮子密钥。量化评估模块定义了命中率、未命中率、泄露带宽、攻击成功率等指标，用于客观度量攻击效果和泄露规模。框架支持一键运行和批量测试。

评估层，在 Linux 平台上对随机噪声注入和缓存行刷新两种防御手段进行对照测试。首先获取默认系统配置下的基线数据，再逐一实验不同强度的缓解措施：CLFLUSH 缓存刷新指令、不同概率的随机噪声注入等策略，记录统计指标变化并分析。通过量化对比不同防御措施下的命中率变化和泄露带宽衰减，评估各方案的实际防护效果和性能开销。三个层次各有分工：理论层推导数学公式，设计

实现层编写代码并进行测试，评估层以实验数据得出量化结论。

1.4 论文组织结构

全文共五章。

第一章绪论。阐述研究背景与选题意义，围绕云多租户环境下的侧信道威胁，梳理 Spectre、Meltdown 等瞬态执行攻击对微架构安全研究格局的冲击。然后从缓存旁路攻击、AES 缓存攻击、微架构侧信道防御、瞬态执行攻击四个方向综述国内外研究现状，点明本文的研究内容与目标。

第二章相关技术背景。阐述本工作依赖的关键技术：CPU 缓存的分层结构与一致性协议、共享内存的实现机制、高精度时间戳的采集方式、AES 分组密码的算法细节与 T 表加速实现，以及 Flush+Reload 攻击的基本原理和几种常见缓解措施。

第三章攻击方法设计与实现。详细阐述攻击方法的设计思路和工程实现：从排除法的数学构造出发，推导单样本分析和多样本累积的统计收敛过程。对 AES-192 和 AES-256，详细推导两阶段密钥恢复——如何复用监控数据恢复倒数第二轮子密钥，再经密钥扩展逆运算反解原始密钥。然后介绍系统架构设计、共享内存实现、量化评估指标体系、缓解措施的工程实现。

第四章实验与分析。展示基准配置下 AES-128/192/256 三种密钥长度的攻击效果，给出命中率、密钥恢复成功率、泄露带宽等指标数据。在不同缓解措施（随机噪声注入、缓存行刷新）下进行对照测试，讨论性能开销与安全性之间的权衡，分析各防御方案的实际防护效果。

第五章总结与展望。总结全文研究工作，展望后续值得深入的方向。

第二章 相关技术背景

Flush+Reload 攻击的运作需要多层机制协同支撑：CPU 缓存管理机制、进程共享内存机制、高精度计时机制，以及 AES 的 T 表实现为何存在信息泄露。本章从硬件到软件，对每一层机制的原理逐一展开。

2.1 CPU 缓存结构

2.1.1 缓存层次结构

CPU 运算速度远高于主存访问速度。以 2020 年代主流桌面处理器为例，CPU 每个时钟周期能完成多条整数运算，一次主存访问却偶尔需要 200 到 400 个周期。这一速度差异是体系结构中不可避免的，缓存即为填补该差距的中间存储层。

现代 x86 处理器普遍采用三级金字塔结构。

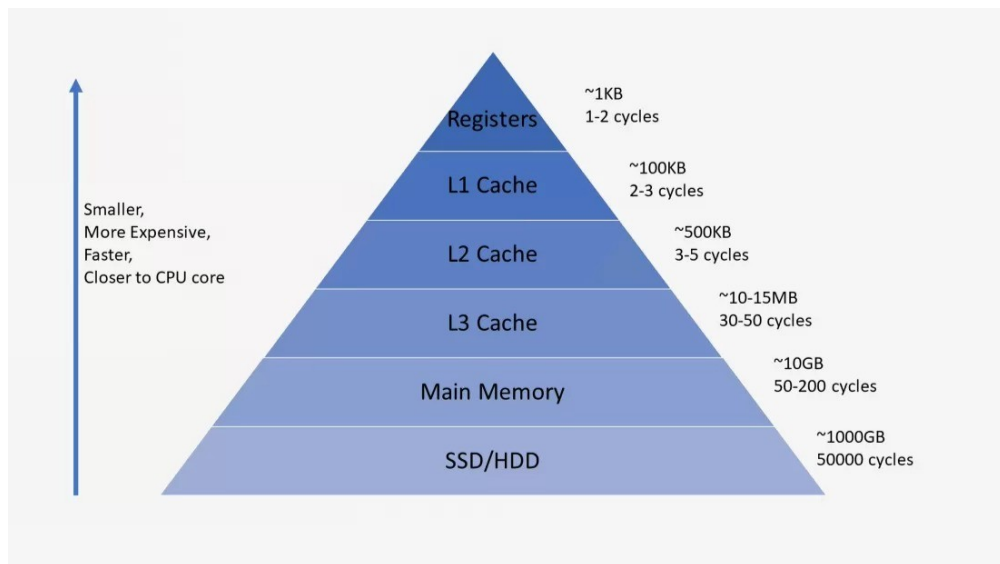


图 2-1 现代 x86 处理器的三级缓存层次结构。每个核心拥有私有的 L1 指令/数据缓存和 L2 缓存，所有核心共享 L3 缓存。从 L1 到主存，容量递增但访问延迟也递增

L1 容量 32–64KB，延迟约 4 周期，分指令缓存（L1I）和数据缓存（L1D），一个核心独占。L2 容量 256KB–1MB，延迟约 12 周期，统一型缓存，同样每核心独占。L3——也叫末级缓存（Last-Level Cache, LLC）——容量从几 MB 到几

十 MB，延迟约 40 周期，所有核心共享。从 L1 到 L3，容量递增但访问延迟也递增，本质上是以空间换取时间。

多级缓存能奏效，依赖于程序的两个规律：空间局部性和时间局部性。空间局部性是指，程序访问了某个地址后，短期内大概率会访问附近的地址——数组遍历、指令顺序执行即为典型示例。时间局部性是指，刚被访问过的数据，短期内很可能再次被访问——循环变量、频繁调用的函数属于此类。

缓存行（Cache Line）的设计直接服务于空间局部性：一次不只载入请求的那个字节，而是将相邻一整块数据一并载入。多级缓存层次则服务于时间局部性：刚使用过的数据留在 L1，稍久未用的回退到 L2 或 L3，再次需要时能快速找回。

2.1.2 缓存行与映射方式

缓存以固定大小的数据块作为管理和传输的基本单位，即缓存行（Cache Line）。x86 处理器这一行是 64 字节。内存地址映射到缓存位置有三种策略。

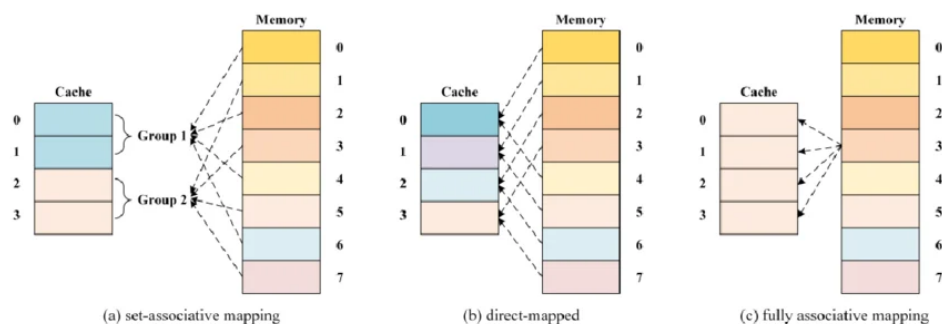


图 2-2 内存地址到缓存行的映射方式对比。左图为直接映射，每个内存块只能映射到唯一缓存行；中图为全相联映射，内存块可放入任意缓存行；右图为组相联映射，内存块先按组号定位，再在组内选择

直接映射：每个内存块只能映射到缓存中唯一的一行。查找速度快，但冲突严重。两个频繁交替访问、又恰好被映射到同一行的内存块，会反复将对方驱逐——这就是所谓的“抖动”。

全相联映射：内存块可以放入缓存中的任意一行。冲突最少，但查找需并行比较所有行标签，硬件成本高，仅适用于小容量缓存（如 TLB）。

组相联映射：将缓存划分为若干组（Set），内存块先按地址（组号）映射到固定组，然后在组内任选一行存放。 n 路组相联即每组有 n 行。 n 越大冲突越少，硬件也越复杂。主流处理器的 L1 一般为 8 路组相联，L3 可达 12–16 路。

2.1.3 缓存替换策略

缓存组已满，但新数据需要载入，则需淘汰旧数据。替换策略决定淘汰哪一行。实际处理器中采用的主要有三种。

LRU：淘汰最近最少使用的那行。理论上最优——它利用了时间局部性，认为最近未使用的短期内也不会再被访问。但严格的 LRU 需为每行维护访问时间戳或位序，路数增多时硬件开销增大。如 16 路组相联的 L3，每组需维护 16 个条目的全序关系，实现代价较大。

伪 LRU (PLRU)：是 LRU 的近似，用一棵二叉树记录访问方向。以 4 路为例，3 个二叉位即可表示 4 行的近似 LRU 顺序：每次访问沿路径翻转位值，替换时沿 0 方向选择。PLRU 将硬件开销从 LRU 的 $O(n \log n)$ 降至 $O(n)$ ，8 路、16 路场景下是主流选择。许多 Intel 处理器的 L2 采用 PLRU，而 L3 则部分采用类 LRU 策略（如 PLRU）或自适应替换策略（如基于 RRIP/DRRIP 的动态插入策略）。

随机替换：顾名思义，随机选一行淘汰。实现最简单，无需任何状态位。表面效果较差，但在工作集远大于缓存容量的场景下，随机替换的期望性能与 LRU 差距不大——冲突驱逐不可避免。部分 ARM 处理器的 L1 采用随机替换。

LRU 和 PLRU 的行为是确定性的。监听方可以通过精心构造的访问序列将目标数据驱逐到指定位置——Prime+Probe 攻击即在此基础上发展而来。随机替换引入了不确定性，攻击难度增大，但从根本上消除侧信道风险是无法实现的，攻击者通过大量重复实验仍能统计出替换规律。

2.1.4 缓存寻址：虚拟索引与物理标签

缓存查找需用地址定位缓存行。此处有一个关键设计选择：索引和标签究竟使用虚拟地址还是物理地址。

虚拟地址是 CPU 发出的地址，物理地址是内存芯片上的实际地址，两者通过页表转换。缓存在地址转换之前还是之后查找，决定了索引和标签的类型。

VIPT (Virtual Index, Physical Tag) 是当前 x86 处理器的主流方案。索引用虚拟地址低位，标签用物理地址高位。由于同一页内虚拟地址低位与物理地址低位相同（页内偏移不变），索引可以与地址转换并行计算，无需等待页表查找完成——缓存访问延迟因此缩短。L1 和 L2 普遍采用 VIPT。VIPT 要求缓存容量不超过页大小乘以相联度。以 4KB 页、8 路组相联为例，L1 最多 32KB，这也是 L1 容量通常定为 32KB 的原因。不过，现代 x86 处理器通过缓存分体或物理索引部分重叠等技术，可以在不产生别名问题的前提下突破这一硬上限，例如部分 AMD 处理器的 64KB 4 路 L1 采用微标签等方案仍能维持 VIPT 语义。

PIPT (Physical Index, Physical Tag)，索引和标签均使用物理地址，必须等

待地址转换完成才能查询缓存。延迟增大，但不存在别名问题——不同虚拟地址映射到同一物理地址时，缓存中的行为是一致的。L3 一般采用 PIPT，因为 L3 容量大，不受 VIPT 的容量限制。

VIVT (Virtual Index, Virtual Tag) 索引和标签都用虚拟地址，查缓存不需要地址转换，延迟最低，但存在别名问题：同一物理地址可能通过不同虚拟地址映射进来，缓存中可能出现多个副本，导致一致性问题。现代 x86 处理器已基本不采用 VIVT。

L3 使用物理地址索引，意味着监听方需要知道目标数据的物理地址才能精确操控缓存状态。不过在共享内存场景下，双方映射同一物理页面，虚拟地址的页内偏移与物理地址的页内偏移一致。监听方仍可准确定位目标缓存行。

2.1.5 包含性与非包含性缓存层次

多级缓存之间存在包含性关系。若 L3 存储了 L1/L2 中所有数据的副本，则称 L3 对 L1/L2 是包含的 (Inclusive)。若 L3 可以不包含 L1/L2 的某些数据，则为非包含 (Non-Inclusive)。若 L3 保证不包含 L1/L2 的数据，则为排他的 (Exclusive)。

Intel 处理器早期多采用包含性 L3，L3 是 L1/L2 的超集。包含性简化了一致性协议，某个核心要修改数据时，仅需检查 L3 标签即可判断其他核心是否持有副本，无需逐个查询每个核心的 L1/L2。需要注意的是，自 Skylake 起的部分 Intel 客户端处理器已转向非包含性 L3 设计，当前 Intel 产品线中包含性与非包含性方案并存。

包含性 L3 还有一个重要特性：CLFLUSH 指令触发一次全局的缓存一致性操作。执行该指令的 CPU 核心会向整个一致性域广播针对目标缓存行的无效化消息，所有核心的私有缓存 (L1/L2) 及共享缓存 (如包含性 L3) 中的该行副本都会被置为 I 状态。若某核心持有 M 状态的脏数据，则会先写回主存再无效化。回顾 Flush+Reload 攻击，其正是依赖于该特性——CLFLUSH 保证目标数据从整个缓存层次中移除，刷新后若重新访问延迟较大，则说明数据确实不在任何缓存级别中。

AMD Zen 架构的 L3 是非包含性的。CLFLUSH 指令定义为全局逐出操作：无论缓存层次的包含策略如何，它都会将目标缓存行从整个缓存一致性域内的所有缓存级别 (L1、L2、L3) 中驱逐。在非包含性缓存上执行 CLFLUSH 时，硬件会通过缓存一致性协议 (如 MOESI) 向所有核心广播无效化探测，不仅清除 L3，也会清除任何核心的 L1/L2 副本。因此，CLFLUSH 在非包含性缓存上依然保证全局逐出语义，Flush+Reload 攻击仍然有效。不过，非包含性缓存层次中 L1/L2 可能持有 L3 中不存在的副本，Victim 进程在 L1/L2 中的数据可能因调度等因素

更快被替换，噪声可能略高于包含性缓存。

2.1.6 缓存一致性协议

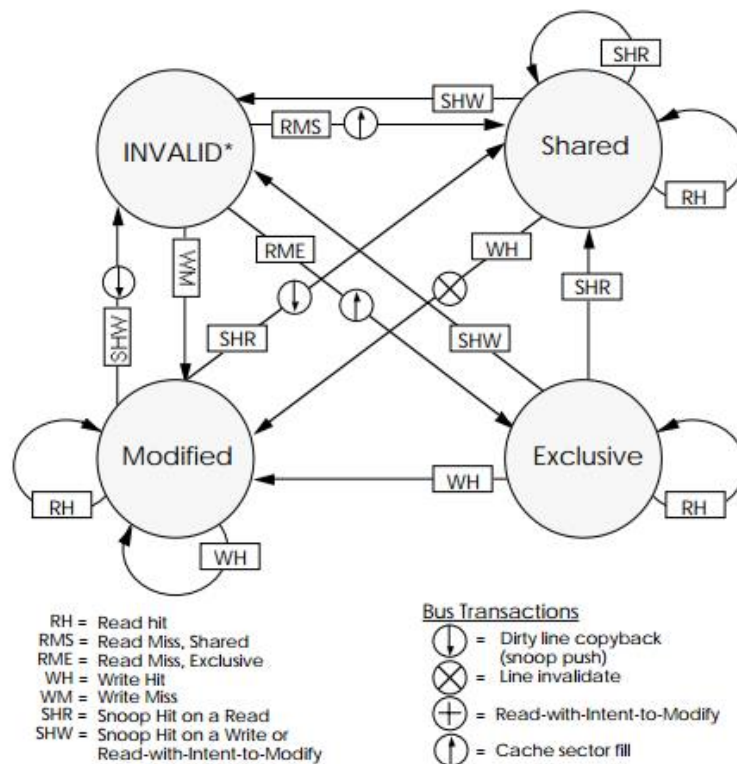
多核芯片上每个核心都可能持有同一内存地址的副本，需要一套机制保证所有核心观测到的数据一致。Intel 处理器采用 MESI 协议维护缓存一致性，为缓存行定义了四种状态：

M (Modified, 已修改)：本核修改了数据，与主存不一致。数据仅存在于本核缓存中。其他核心请求该数据时，本核必须将修改后的数据写回主存，或直接转发给请求方。

E (Exclusive, 独占)：缓存行数据与主存一致，且仅存在于本核缓存中。本核可直接修改——修改后状态转为 M。

S (Shared, 共享)：数据与主存一致，可能在多个核心的缓存中同时存在副本。任一核心欲修改该行，需先通过总线通知其他核心将各自副本置为 I。

I (Invalid, 无效)：该缓存行为空，访问该行一定触发缓存未命中。



MESI cache coherency protocol

图 2-3 MESI 缓存一致性协议的状态转换图。M (Modified) 表示已修改，E (Exclusive) 表示独占，S (Shared) 表示共享，I (Invalid) 表示无效。箭头标注了触发状态转换的事件类型

MESI 协议通过总线上的监听（Snooping）机制运作：每个核心的缓存控制器监听总线上的事务，根据事务类型和本地缓存行的状态执行相应动作。状态转换的关键路径包括：核心读未命中且无其他核心持有该行时，从主存加载并标记为 E；核心读未命中但其他核心持有 S 副本时，从主存加载并标记为 S；核心写命中时，若状态为 M 或 E 直接写入并标记为 M，若状态为 S 则先发出无效化请求（Invalidate），等其他核心将副本置为 I 后再写入并标记为 M。

2.1.7 MESI 协议如何使跨核攻击成为可能

MESI 协议的跨核无效化机制是 Flush+Reload 攻击的协议基础。考虑以下场景：监听方运行于核心 0，被攻击方运行于核心 1。

监听方用 CLFLUSH 将目标缓存行从所有缓存级别驱逐，该行在两个核心的缓存中均变为 I 状态。之后若被攻击方（核心 1）访问了该地址，缓存控制器会从主存或 L3 将数据载入核心 1 的缓存，该行在核心 1 变为 E 或 S 状态。若 L3 是包含性的，同一行也会出现在 L3 中。监听方（核心 0）随后重新访问该地址，即可发现该行已在 L3 中（状态为 S），访问延迟远低于从主存加载——此即缓存命中。反之，若被攻击方未访问该地址，该行仍不在任何缓存中，监听方的访问需通过主存完成，延迟显著增大。

监听方从未与被攻击方直接通信，也没有突破任何权限隔离。其仅测量了自身访问共享内存的时间。但 MESI 协议保证了一点：被攻击方对共享数据的访问会改变缓存状态，缓存状态的变化可通过时间差被观测到。这就是跨核侧信道攻击的原理。

2.1.8 缓存访问时间差异

缓存命中和未命中的延迟差异显著。以 Intel Core i5-1135G7 为例：L1D 命中约 5 周期（约 2.1ns@2.4GHz），L2 命中约 14 周期（约 5.8ns），L3 命中约 50 周期（约 20.8ns），主存访问约 300 周期（约 125ns）。命中与未命中之间存在 100 到 300 周期的稳定差距。Flush+Reload 攻击即将该时间差距作为观测信号。

2.2 共享内存机制

2.2.1 共享内存概述

共享内存使多个进程可访问同一块物理内存。它是高效的 IPC 机制——数据无需在内核缓冲区和用户缓冲区之间反复拷贝，进程直接读写映射到自身地址空间的内存页即可。Linux 支持三种共享内存方式：System V（shmget/shmat）、

POSIX (`shm_open/mmap`)、内存映射文件 (`open/mmap`)。

2.2.2 POSIX 共享内存 API

POSIX 共享内存是本文采用的机制，核心 API 包括：

`shm_open()` 创建或打开一个共享内存对象，参数包括名称、标志位 (`O_RDWR` 等) 和权限模式，返回一个文件描述符。名称以斜杠开头 (如 `"/aes_ttables"`)，在同一系统内全局可见。

`ftruncate()` 设置共享内存对象的大小。新创建的共享内存对象初始大小为 0，必须通过 `ftruncate` 扩展到所需容量。

`mmap()` 将共享内存对象映射到进程的虚拟地址空间，返回映射区域的起始地址。关键参数包括：保护模式 (`PROT_READ | PROT_WRITE`)、映射标志 (`MAP_SHARED` 表示修改对其他进程可见)、文件描述符和偏移量。映射完成后，进程可以像操作普通内存一样读写共享区域。

`shm_unlink()` 删除共享内存对象的名称。名称删除后，已经映射的进程仍可继续访问，但新的 `shm_open` 无法再找到该对象。当所有映射都解除后，共享内存对象自动销毁。

本文的方案是：攻击者进程用 `shm_open` 打开共享内存对象，`mmap` 将 T 表数据映射到自身地址空间；受害者进程同样用 `shm_open` 和 `mmap` 映射同一块共享内存。两个进程的虚拟地址不同，但底层指向同一块物理内存。

2.2.3 写时复制与代码段共享

Linux 的 `fork()` 采用写时复制 (Copy-on-Write, CoW) 机制：子进程创建时不立即复制父进程的内存页，而是让父子共享同一物理页，并全部标记为只读。任一方先写入时，CPU 触发页故障，内核此时才复制该页、修改页表使写方指向新物理页。CoW 大幅降低了 `fork` 的开销，也带来一个值得注意的问题——`fork` 后的父子进程在写入前共享物理内存，这本身就构成侧信道风险。

动态链接库的代码段是共享内存的另一个典型场景，且不受 CoW 影响。多个进程加载同一个动态库，操作系统只需将库代码载入物理内存一次，所有进程共享同一份物理映射。代码段被标记为只读 (`PROT_READ | PROT_EXEC`)，写操作直接触发段错误而非 CoW。这意味着代码段的物理页永远不会被复制，始终为所有进程共享的同一份。

此处需注意，这为 Flush+Reload 探测提供了有利条件——监听方可以加载与被攻击方相同的动态库，监视被攻击方对库代码的访问模式，无需考虑 CoW 机制导致物理页分离。

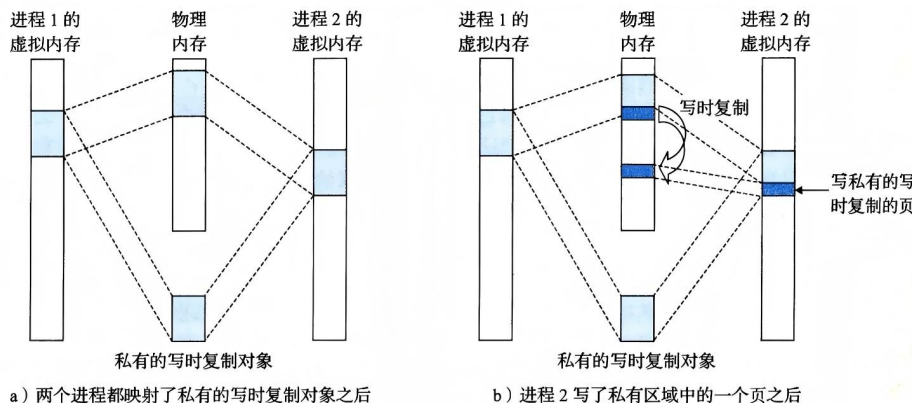


图 2-4 共享内存与写时复制 (CoW) 机制示意图。左图展示 fork 后父子进程初始共享物理页面；右图展示任一方执行写操作时触发页故障，操作系统为写方创建独立副本的过程

数据段则不同。若被攻击方修改了数据段中的变量，CoW 机制将创建物理页副本，双方从此访问不同物理页，缓存侧信道攻击即失效。因此 Flush+Reload 一般选择监视只读的代码段，或显式创建的共享内存区域。

2.2.4 虚拟化环境中的内存去重

共享内存存在虚拟化场景下有更广泛的存在形式。云平台上的不同虚拟机运行于同一台物理主机上，通过 Hypervisor 管理物理内存分配。为了提升内存利用率，Hypervisor 开启 KSM (Kernel Same-page Merging)：定期扫描各个虚拟机的内存页，找出内容相同的页面，将它们合并为同一物理页的映射，标记为 CoW。若某台虚拟机修改了该合并页面，CoW 机制再创建独立副本。

KSM 在云环境中带来了显著的内存节省，但也引入了严重的安全风险。不同租户的虚拟机可能装载相同的 OS 内核、相同的库文件——KSM 将这些页面合并后，不同虚拟机的物理页即实现了共享。攻击者虚拟机可以借助该共享关系，对同宿主机上的其他租户虚拟机发起跨 VM 的 Flush+Reload 探测。Zhang 等人在 PaaS 云环境中验证了该攻击的可行性，成功跨租户恢复了 AES 密钥。

内存去重是 KSM 的推广概念，不仅合并相同页面，还可能合并部分相同的页面。Windows 和 ESXi 也有类似的去重机制。

安全问题明确：内存去重将共享内存的攻击面放大了。两个进程即使没有显式建立共享内存，只要恰好存在内容相同的内存页，去重机制便隐式建立了共享关系。Benger 等人的研究表明，即使仅有少量共享页面，也足以支撑高精度的缓存侧信道攻击。

2.3 时间测量方法

2.3.1 时间戳计数器

x86 处理器提供高精度时间戳计数器，通过 RDTSC/RDTSCP 指令读取。TSC 以固定频率递增，每个时钟周期加 1，提供纳秒级的时间分辨率。实际攻击中用的时间测量核心代码：

```
static inline uint64_t rdtsc() {
    uint64_t a, d;
    asm volatile ("rdtsc" : "=a" (a), "=d" (d));
    return (d << 32) | a;
}

static inline void flush_cache_line(const void *addr) {
    asm volatile ("clflush 0(%0)" :: "r" (addr) : "memory");
}
```

2.3.2 RDTSC 与 RDTSCP 的区别

RDTSC 和 RDTSCP 都读取 TSC，但行为上有一个关键差别。RDTSC 不是串行化指令，处理器可以在 RDTSC 前后重排其他指令——这样读取的时间戳便无法精确反映 RDTSC 前后代码的实际执行时间。例如处理器可能将 RDTSC 之后的访存指令提前执行，测出的时间就会偏短。实际中常在 RDTSC 前后插入 LFENCE 指令来阻止这种重排。

RDTSCP 是部分串行化：它保证 RDTSCP 之前的指令全部执行完毕后才读取 TSC，但之后的指令仍可能被提前执行。因此 RDTSCP 之后加一条 LFENCE 即可实现完整序列化，比 RDTSC+ 双 LFENCE 效率更高。需要注意的是，LFENCE 在 Intel 上是序列化屏障，在较老的 AMD 处理器上行为可能不同，攻击代码通常需要考虑平台差异。在 Flush+Reload 攻击里，RDTSCP 是首选的时间测量指令。

另外，现代 Intel 处理器支持恒定 TSC (Invariant TSC)，TSC 频率不随 CPU 动态调频变化。这在 Nehalem 及之后的架构上是默认行为，保证了时间测量在不同频率下的一致性。若 TSC 不恒定，CPU 降频时 TSC 频率降低，测量的周期数偏大，阈值将直接失效。

2.3.3 用户态时间测量的精度问题

在用户态进行高精度时间测量会面临多种干扰因素。

上下文切换是最大的干扰源。OS 调度器可能在中断、时间片耗尽、系统调用等时刻将当前进程换出。一次上下文切换耗时数千至数万周期，远超缓存命中与未命中的时间差。测量过程中若进程被切换，测得的时间便混入了调度开销，无法反映真实的缓存访问延迟。

中断同样棘手。硬件中断——定时器、网络、键盘等——可在任意时刻打断当前进程，处理完毕后才恢复执行。中断处理的时间虽短于上下文切换，但也在数百至数千周期量级，足以干扰时间测量。

预取是硬件层面的适得其反。硬件预取器发现顺序访问模式时，会提前将后续缓存行载入缓存。若预取器恰好将探测方即将测量的缓存行载入，本应“未命中”的测量便变为“命中”——假阳性噪声由此产生。

如何应对上述干扰？

绑定 CPU 核心可以防止进程迁移导致的缓存冷启动，这是用户态最容易实现的。关闭中断需要内核权限，用户态无法实现。设置实时调度策略能减少调度干扰，同样受权限限制。重复测量取统计量——用中位数而非均值来消除异常值的影响——是数据层面的补救。本文采用的方法是核心绑定加统计滤波，在用户态条件下尽可能提高测量精度。唐博文等人对时间侧信道的攻防技术做过全面综述，梳理了不同防御机制的保护能力和性能瓶颈。

2.3.4 时间阈值确定

区分缓存命中和未命中需要一个时间阈值。本文采用中位数方法：分别测量缓存命中和未命中时间的中位数 M_{hit} 和 M_{miss} ，取 $T = (M_{hit} + M_{miss})/2$ 作为阈值。典型值 $M_{hit} \approx 80$ 周期， $M_{miss} \approx 300$ 周期， $T \approx 190$ 周期。

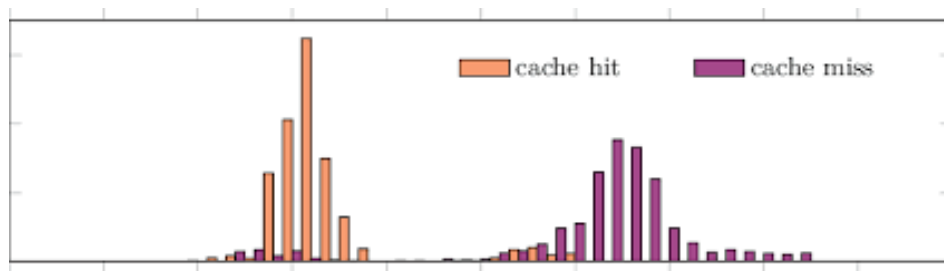


图 2-5 缓存命中与未命中的访问时间分布对比。左峰为缓存命中时间分布（低延迟），右峰为未命中时间分布（高延迟），两者之间存在明显的双峰特征。中间虚线标注分类阈值 T ，用于区分两种状态

为何采用中位数？缓存时间测量中异常值过多。上下文切换可使某次测量增至数万周期，硬件预取可使某次未命中测量异常偏小。均值对异常值极度敏感——一个极端异常值即可使均值产生偏差，导致阈值不准确。中位数则不同：只

要异常值不超过样本量的一半，中位数基本不受影响。缓存时间测量场景下，异常样本通常占总样本的 5% 到 15%，远低于 50% 的临界点，中位数方法更为稳健。

校准过程：首先对共享内存中的目标地址执行一次访问，确保数据在缓存中，然后连续测量 10000 次命中时间；接着用 CLFLUSH 驱逐缓存行，连续测量 10000 次未命中时间；分别计算两组数据的中位数，取平均值作为阈值。整个校准过程在攻击开始前执行。

2.4 AES 加密算法原理

2.4.1 AES 算法概述

高级加密标准（Advanced Encryption Standard, AES）是由 Rijndael 算法演化而来的对称分组加密算法，2001 年由 NIST 正式发布为 FIPS PUB 197。AES 支持 128 位、192 位、256 位三种密钥长度，对应加密轮数 10、12、14 轮，分组长度固定 128 位。AES 的安全性依赖两条设计原则：混淆（Confusion）——使密文与密钥之间的统计关系尽可能复杂；扩散（Diffusion）——使明文的每一位影响密文的尽可能多位。

2.4.2 AES 数学基础

AES 的运算在有限域 $GF(2^8)$ 上进行。有限域（也称伽罗瓦域）是包含有限个元素的代数结构，支持加法、减法、乘法和除法（除 0 外）四种运算，且满足交换律、结合律和分配律。 $GF(2^8)$ 包含 256 个元素，每个元素可以表示为一个字节（8 位），也可以表示为系数在 $GF(2)$ （即 $\{0, 1\}$ ，加法为异或）上的 7 次多项式：

$$a(x) = a_7x^7 + a_6x^6 + a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0 \quad (2-1)$$

例如，字节 0x57 对应多项式 $x^6 + x^4 + x^2 + x + 1$ ，字节 0x83 对应多项式 $x^7 + x + 1$ 。

$GF(2^8)$ 上的加法就是系数按位异或。例如 $0x57 \oplus 0x83 = 0xD4$ ，对应多项式加法 $(x^6 + x^4 + x^2 + x + 1) + (x^7 + x + 1) = x^7 + x^6 + x^4 + x^2$ ，系数模 2 后无进位。

乘法是多项式乘法模不可约多项式 $m(x) = x^8 + x^4 + x^3 + x + 1$ （即 0x11B）。不可约多项式的作用类似于素数——它不能在 $GF(2)$ 上分解为更低次多项式的乘积，保证了 $GF(2^8)$ 中每个非零元素都有乘法逆元。乘法运算的步骤：先将两个多项式按普通多项式乘法相乘（系数模 2），然后对 $m(x)$ 取模（即多项式除法取余数）。例如 $0x57 \times 0x83$ ：

先算 $0x57 \times 0x02 = 0xAE$ （左移 1 位）， $0x57 \times 0x80 = 0x57$ 左移 7 位，需要多次模 $m(x)$ 约化。实际计算中，乘以 x （即 $0x02$ ）是最常用的基本操作，称为 *xtime*：

$$xtime(a) = \begin{cases} a \ll 1 & \text{if } a_7 = 0 \\ (a \ll 1) \oplus 0x1B & \text{if } a_7 = 1 \end{cases} \quad (2-2)$$

当 a 的最高位为 0 时，左移 1 位不会溢出字节范围；当最高位为 1 时，左移后产生 x^8 项，需要异或 $0x1B$ （即 $m(x)$ 去掉 x^8 项后的低 8 位）来约化。任意乘法都可以分解为若干次 *xtime* 和异或。例如 $0x57 \times 0x13 = 0x57 \times (0x01 \oplus 0x02 \oplus 0x10) = 0x57 \oplus xtime(0x57) \oplus xtime^4(0x57)$ 。

2.4.3 AES 状态矩阵

AES 把 128 位输入排成 4×4 的字节矩阵，按列优先顺序填充：

$$State = \begin{pmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{pmatrix} \quad (2-3)$$

其中 b_0 是输入的第一个字节， b_{15} 是最后一个字节。状态矩阵的每一列对应一个 32 位字，加密操作在状态矩阵上逐字节或逐列进行。

2.4.4 AES 轮函数

AES 加密由一个初始轮密钥加、若干标准轮和一个最终轮组成。标准轮包含 SubBytes、ShiftRows、MixColumns、AddRoundKey 四个操作，最终轮没有 MixColumns。初始轮密钥加将明文与第一个轮密钥异或，为后续轮变换提供密钥依赖的输入。

SubBytes 变换。SubBytes 用 S 盒对状态矩阵逐字节做非线性替换，这是 AES 中唯一的非线性操作。S 盒的构造分两阶段：先在 $GF(2^8)$ 上求输入字节的乘法逆元（0 映射到 0），再对结果做仿射变换：

$$S[x] = A \cdot (x^{-1} \bmod m(x)) + b \quad (2-4)$$

其中 A 是 $GF(2)$ 上的 8×8 可逆矩阵， $b = (1, 1, 0, 0, 0, 1, 1, 0)^T$ 。该构造同时满足了两个需求：求逆提供了非线性，仿射变换打乱了代数结构。其结果是差分分析和线性分析均难以奏效。S 盒的 256 个值是固定的，可以预计算为查找表。例如 $S[0x00] = 0x63$ ， $S[0x01] = 0x7C$ ， $S[0x53] = 0xED$ ， $S[0xFF] = 0x16$ 。

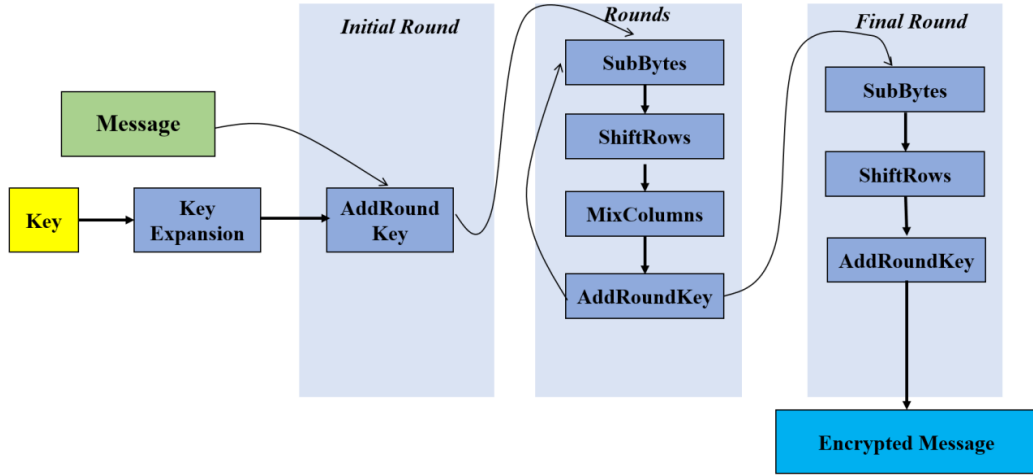


图 2-6 AES 加密算法的整体流程图。从明文出发，先经过初始轮密钥加（AddRoundKey），然后执行 $N_r - 1$ 次标准轮（每轮包含 SubBytes、ShiftRows、MixColumns、AddRoundKey 四个操作），最后执行不含 MixColumns 的最终轮得到密文

ShiftRows 变换。ShiftRows 对状态矩阵的行做循环左移，行号就是移位量：第 0 行不移，第 1 行移 1 字节，第 2 行移 2 字节，第 3 行移 3 字节：

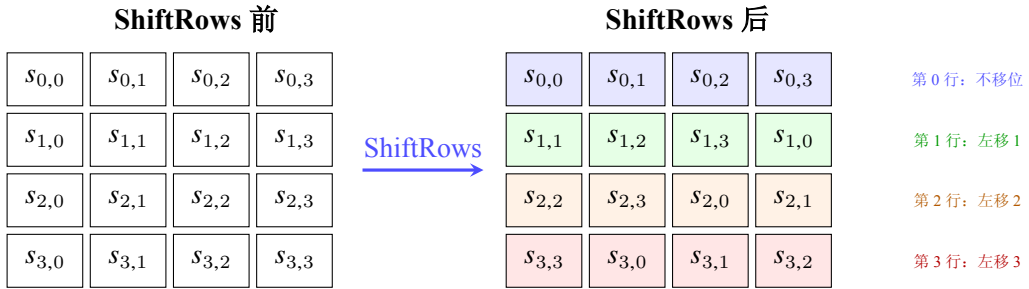


图 2-7 AES ShiftRows 变换示意图。状态矩阵的每一行按行号进行循环左移：第 0 行不移位，第 1 行左移 1 字节，第 2 行左移 2 字节，第 3 行左移 3 字节。不同颜色标注各行位移量

$$\begin{pmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,1} & s_{1,2} & s_{1,3} & s_{1,0} \\ s_{2,2} & s_{2,3} & s_{2,0} & s_{2,1} \\ s_{3,3} & s_{3,0} & s_{3,1} & s_{3,2} \end{pmatrix} \quad (2-5)$$

ShiftRows 的作用是列间扩散：将同一列的字节分散到不同列中，使 MixColumns 能够跨列混合数据。若缺少 ShiftRows，每列的 MixColumns 仅影响本列，攻击者可以逐列独立分析。

MixColumns 变换。MixColumns 对状态矩阵的每一列做 $GF(2^8)$ 上的矩阵乘法，使用固定矩阵：

$$\begin{pmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{pmatrix} = \begin{pmatrix} 0x02 & 0x03 & 0x01 & 0x01 \\ 0x01 & 0x02 & 0x03 & 0x01 \\ 0x01 & 0x01 & 0x02 & 0x03 \\ 0x03 & 0x01 & 0x01 & 0x02 \end{pmatrix} \begin{pmatrix} s_{0,j} \\ s_{1,j} \\ s_{2,j} \\ s_{3,j} \end{pmatrix} \quad (2-6)$$

矩阵中的乘法在 $GF(2^8)$ 上进行，加法为异或。系数 0x01、0x02、0x03 分别对应乘 1、乘 x 、乘 $(x+1)$ 。这个矩阵是 MDS (Maximum Distance Separable) 矩阵，保证了扩散的最优性——输入中任意改变 1 个字节，输出中至少改变 4 个字节。

AddRoundKey 变换。AddRoundKey 将状态矩阵与当前轮密钥逐字节异或：

$$s'_{i,j} = s_{i,j} \oplus k_{i,j} \quad (2-7)$$

这是轮函数中唯一引入密钥信息的步骤。异或运算的可逆性保证了解密时只需再次异或同一轮密钥即可恢复原始状态。

2.4.5 AES 一轮运算示例

以一个简化的例子说明一轮 AES 运算的过程。设某轮输入状态矩阵的第 0 列为 $(0xDB, 0x13, 0x53, 0x22)^T$ ，轮密钥第 0 列为 $(0xA0, 0xE0, 0xC0, 0xF0)^T$ 。

SubBytes: 查 S 盒替换每个字节。 $S[0xDB] = 0xB9$, $S[0x13] = 0x7D$, $S[0x53] = 0xED$, $S[0x22] = 0x93$ 。替换后第 0 列变为 $(0xB9, 0x7D, 0xED, 0x93)^T$ 。

ShiftRows: 第 0 行不移位，第 0 列的第 0 行字节 0xB9 位置不变。但第 1 行左移 1 字节，原来第 1 列的字节移到第 0 列；第 2 行左移 2 字节，原来第 2 列的字节移到第 0 列；第 3 行左移 3 字节，原来第 3 列的字节移到第 0 列。因此 ShiftRows 后第 0 列的 4 个字节来自不同原始列。

MixColumns: 对 ShiftRows 后的第 0 列做矩阵乘法。设 ShiftRows 后第 0 列为 $(s_0, s_1, s_2, s_3)^T$ ，则：

$$s'_0 = 0x02 \cdot s_0 \oplus 0x03 \cdot s_1 \oplus s_2 \oplus s_3 \quad (2-8)$$

其中 $0x02 \cdot s_0 = xtime(s_0)$, $0x03 \cdot s_1 = xtime(s_1) \oplus s_1$ 。

AddRoundKey: 将 MixColumns 的输出与轮密钥异或， $s''_0 = s'_0 \oplus 0xA0$ ，以此类推。

这个例子展示了一轮运算中数据如何经过非线性 S 盒替换、行位移、列混淆和轮密钥加四个步骤的逐层变换。

2.4.6 AES 密钥扩展

密钥扩展把初始密钥分成 N_k 个 32 位字 ($N_k = 4/6/8$ 对应 AES-128/192/256), 递归生成后续字:

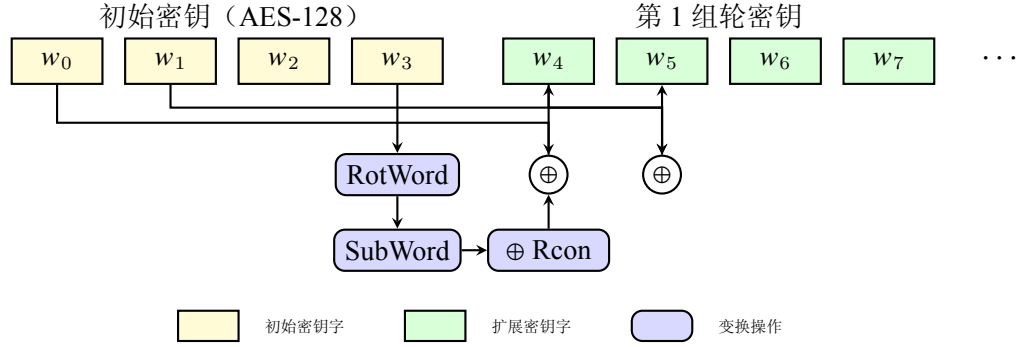


图 2-8 AES-128 密钥扩展流程示意图。初始密钥的 4 个字 w_0-w_3 通过 $RotWord$ 、 $SubWord$ 和 $Rcon$ 变换递归生成后续字。当 $i \bmod 4 = 0$ 时, $w_i = w_{i-4} \oplus SubWord(RotWord(w_{i-1})) \oplus Rcon[i/4]$; 否则 $w_i = w_{i-4} \oplus w_{i-1}$

$$w_i = \begin{cases} k_i & \text{if } i < N_k \\ w_{i-N_k} \oplus SubWord(RotWord(w_{i-1})) \oplus Rcon[i/N_k] & \text{if } i \geq N_k \text{ and } i \bmod N_k = 0 \\ w_{i-N_k} \oplus SubWord(w_{i-1}) & \text{if } N_k > 6 \text{ and } i \bmod N_k = 4 \\ w_{i-N_k} \oplus w_{i-1} & \text{otherwise} \end{cases} \quad (2-9)$$

其中 $RotWord$ 将 32 位字循环左移一个字节: $(a_0, a_1, a_2, a_3) \rightarrow (a_1, a_2, a_3, a_0)$ 。 $SubWord$ 对 32 位字的每个字节分别做 S 盒替换。 $Rcon[r]$ 是轮常量, 其第一个字节为 x^{r-1} 在 $GF(2^8)$ 上的值 ($Rcon[1] = 0x01000000$, $Rcon[2] = 0x02000000$, $Rcon[3] = 0x04000000$, ...), 其余三个字节为 0。

对 AES-128 ($N_k = 4$), 初始密钥产生 w_0, w_1, w_2, w_3 , 然后每 4 个字为一组扩展。当 $i \bmod 4 = 0$ 时, $w_i = w_{i-4} \oplus SubWord(RotWord(w_{i-1})) \oplus Rcon[i/4]$; 否则 $w_i = w_{i-4} \oplus w_{i-1}$ 。总共生成 44 个字 (11 组, 对应初始轮密钥加和 10 轮加密)。

对 AES-192 ($N_k = 6$), 初始密钥产生 $w_0, \dots, w_5$, 每 6 个字为一组扩展。当 $i \bmod 6 = 0$ 时使用 $SubWord(RotWord(\cdot))$ 和 $Rcon$ 分支; 否则 $w_i = w_{i-6} \oplus w_{i-1}$ 。总共生成 52 个字 (对应 12 轮加密)。

对 AES-256 ($N_k = 8$), 初始密钥产生 $w_0, \dots, w_7$, 每 8 个字为一组扩展。当 $i \bmod 8 = 0$ 时使用 $SubWord(RotWord(\cdot))$ 和 $Rcon$ 分支; 当 $i \bmod 8 = 4$ 时另有特殊分支:

$$w_i = w_{i-8} \oplus \text{SubWord}(w_{i-1}) \quad (2-10)$$

该额外分支增加了 AES-256 密钥扩展的非线性度，是 AES-256 相对于 AES-128 的一个安全增强，但也使得其密钥扩展过程并非处处均匀。总共生成 60 个字（对应 14 轮加密）。

密钥扩展的这一特性，使攻击者可以通过恢复最后一轮密钥逆向推出原始密钥。以 AES-128 为例，恢复 K_{10} （即 $w_{40}, w_{41}, w_{42}, w_{43}$ ）后，利用 $w_{i-4} = w_i \oplus w_{i-1}$ （ $i \bmod 4 \neq 0$ 时）或 $w_{i-4} = w_i \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/4]$ （ $i \bmod 4 = 0$ 时）逐步逆推到 w_0, w_1, w_2, w_3 ，即原始密钥 K_0 。

2.4.7 AES T 表实现

T 表将 SubBytes、ShiftRows 和 MixColumns 三个操作合并为预计算查找表。利用该优化后，一轮加密中每个输出字节的计算从 4 次 S 盒查表 + 4 次域乘法 + 1 次移位重排，简化为 4 次 T 表查表和 3 次异或操作。四个 T 表的定义：

$$T_0[x] = \begin{pmatrix} S[x] \cdot 0x02 \\ S[x] \\ S[x] \\ S[x] \cdot 0x03 \end{pmatrix}, \quad T_1[x] = \begin{pmatrix} S[x] \cdot 0x03 \\ S[x] \cdot 0x02 \\ S[x] \\ S[x] \end{pmatrix} \quad (2-11)$$

$$T_2[x] = \begin{pmatrix} S[x] \\ S[x] \cdot 0x03 \\ S[x] \cdot 0x02 \\ S[x] \end{pmatrix}, \quad T_3[x] = \begin{pmatrix} S[x] \\ S[x] \\ S[x] \cdot 0x03 \\ S[x] \cdot 0x02 \end{pmatrix} \quad (2-12)$$

每个 $T_i[x]$ 是一个 32 位值，由 S 盒输出 $S[x]$ 乘以 MixColumns 矩阵的第 i 列系数得到。 T_0 对应 MixColumns 矩阵的第 0 列， T_1 对应第 1 列，依此类推。ShiftRows 的字节重排通过选择不同的 T 表来实现——第 i 行字节统一查 T_i 表，ShiftRows 的行偏移则体现在输入字节的列索引上（第 i 行取列 $j + i \bmod 4$ 的字节）。

采用 T 表后，一轮加密可以写成：

$$\begin{pmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{pmatrix} = T_0[s_{0,j}] \oplus T_1[s_{1,j+1}] \oplus T_2[s_{2,j+2}] \oplus T_3[s_{3,j+3}] \oplus k_j \quad (2-13)$$

换言之，每轮 4 次 S 盒查表 + 4 次 MixColumns 矩阵乘法 + 1 次 ShiftRows 重排 + 1 次 AddRoundKey，被合并成了 16 次 T 表查表 + 12 次 32 位异或 + 4 次轮密

钥异或。这种查表驱动的实现在软件中远快于逐字节做域运算，但代价也很明确：T 表访问模式依赖于密钥相关的索引值，攻击者可以通过观测缓存状态推断索引信息。

2.4.8 T 表与缓存行的映射关系

每个 T 表含 256 个 32 位入口，共 1024 字节。一个 T 表共 256 项，索引为 0x00 到 0xFF。一个缓存行（64 字节）可以存放 16 个 4 字节的表项。因此，索引 0x00-0x0F 在第 0 缓存行，0x10-0x1F 在第 1 缓存行，以此类推，共 16 个缓存行。对于 64 字节的缓存行，每个缓存行存 16 个入口（ $16 \times 4 = 64$ 字节），T 表索引的高 4 位决定缓存行位置：

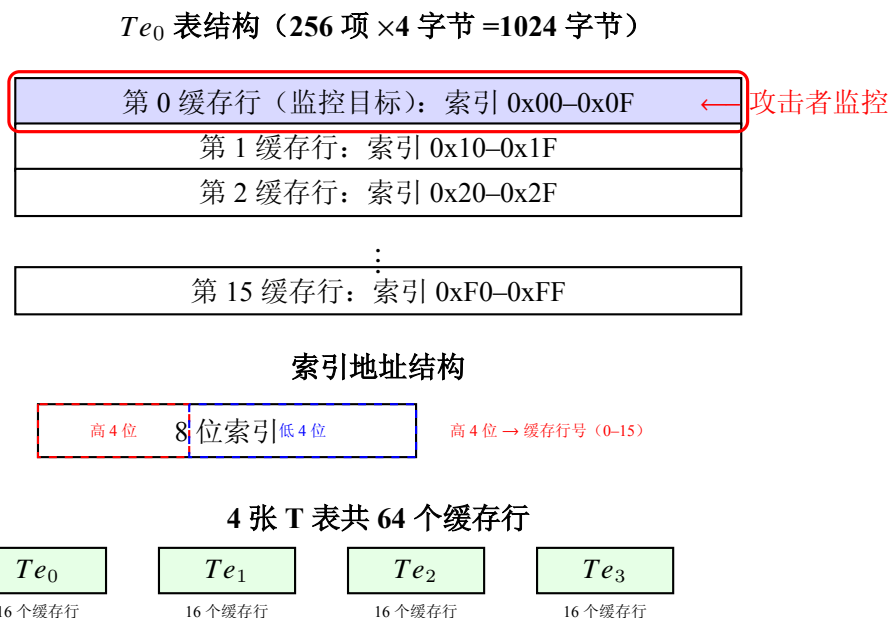


图 2-9 AES T 表结构与缓存行映射关系。每个 T 表含 256 个 32 位表项，共 1024 字节，按每 16 项一组划分为 16 个缓存行。索引的高 4 位决定缓存行号。攻击者仅监控每张 T 表的第 0 号缓存行（红色标注），共 4 个监控点

$$CacheLine(T_i[x]) = \lfloor x/16 \rfloor = x \gg 4 \quad (2-14)$$

攻击者以缓存行为单位监视 T 表访问，每次即可获取索引的高 4 位——即 4 比特信息。攻击者观察到对某个缓存行的访问，即可将查表索引的高 4 位信息确定下来。4 张 T 表共 64 个缓存行，本文通常仅监控每张表的第 0 号缓存行（对应索引 0-15），共 4 个监控点。该策略降低了 cflflush 开销和测量噪声，同时利用排除法仍可逐步收敛到正确密钥。

基于查找表的 AES 实现是缓存侧信道攻击的核心目标，攻击目标是融合了

MixColumns 的 T_e 表，OpenSSL、Libgcrypt 等主流密码库均采用 T 表优化，大量已部署软件面临侧信道风险。替代方案有位切片（Bitsliced）实现：将多个数据块打包到 SIMD 寄存器中并行处理，直接去除查表操作，从源头切断缓存侧信道的条件。但位切片需同时处理多个数据块，单块加密场景无法使用。

2.5 Flush+Reload 攻击原理

2.5.1 攻击模型

Flush+Reload 攻击的成立需要满足若干前提：第一，探测方和被攻击方必须共享内存——双方映射同一块物理内存，缓存状态才能共享，时间测量才能反映被攻击方的行为。第二，探测方需能执行 CLFLUSH 指令。这是 x86 的缓存行逐出指令。第三，需要高精度时间测量，依赖 RDTSC/RDTSCP。最后，探测方需要知道共享内存中目标数据的地址布局——如 T 表的偏移量。在动态库场景下，加载同一库即可计算偏移；自定义共享内存场景下，偏移量可自行确定。

2.5.2 攻击流程

Flush+Reload 攻击的核心是一个四步循环。

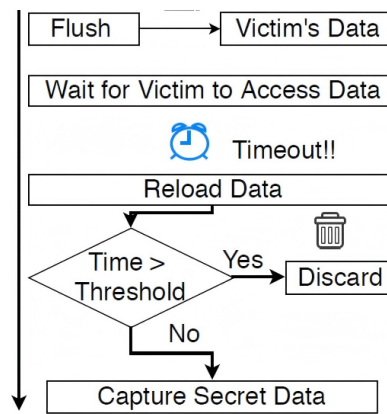


图 2-10 Flush+Reload 攻击的四步循环流程图。攻击者通过反复执行刷新（Flush）→ 等待（Wait）→ 重载（Reload）→ 推断（Infer）的循环，收集缓存访问模式的统计信息，最终推断出受害者的密钥信息

第 1 步：刷新（Flush）——CLFLUSH 将目标缓存行从所有缓存级别中逐出。CLFLUSH 触发 MESI 协议的无效化操作，目标行在所有缓存层级均置为 I。在包含性缓存层次中，L3 中的副本也被清除。

第 2 步：等待（Wait）——等待被攻击方执行。若被攻击方访问了被刷新的地址，数据将进入 Victim 进程的 L1/L2 和共享 L3，该行在 L3 中处于 Shared 状

态。若未访问，缓存行仍为 I。

第 3 步：重载 (Reload) ——重新访问目标地址，测量时间。数据在 L3 (被攻击方访问过) 就约 40-80 周期 (命中)；不在任何缓存 (没访问过) 就约 200-400 周期 (未命中)。

第 4 步：推断 (Infer) ——根据缓存状态推测被攻击方的行为。时间低于阈值 T，判为命中，说明被攻击方访问了目标缓存行。高于阈值判为未命中。

四步反复循环，累积足够样本即可推断出访问模式，进而恢复密钥等敏感信息。

每次循环的时间开销主要取决于等待阶段，攻击者需要等受害者完成一次完整的加密操作。在同步攻击 (攻击者触发受害者加密) 中，等待时间可以精确控制；在异步攻击中，等待时间不确定，需要通过多次采样来补偿。

2.5.3 Flush+Flush 变体

Gruss 等人提出的 Flush+Flush 是 Flush+Reload 的变体。它不测量数据访问时间，而是测量 CLFLUSH 指令本身的执行时间。其原理在于：目标缓存行若在缓存中 (脏或干净)，CLFLUSH 需执行无效化操作，耗时长 (约 200 周期)；若不在缓存中，CLFLUSH 几乎无需操作，耗时很短 (约 40 周期)。

Flush+Flush 与 Flush+Reload 相比有几个值得注意的差异。它不需要实际访问目标数据，从而避开了数据访问可能触发的异常和审计，降低了被检测的风险。它也不改变缓存状态——CLFLUSH 之后缓存行确实被逐出了，但探测方不重载，不在缓存中留下自身的访问痕迹。但也存在代价：CLFLUSH 的执行时间差异通常小于 Flush+Reload 的访问时间差异 (在典型 Intel 平台上约 160 周期 vs Flush+Reload 的 200+ 周期)，信噪比稍低，但该差值因微架构和内存状态而异，不同平台可能接近或超过 100 周期；而且 CLFLUSH 会破坏缓存状态，可能影响 Victim 进程的性能。

2.5.4 与其他缓存攻击方法的比较

缓存侧信道攻击有多种实现方法，Flush+Reload 仅为其中之一。下面对比三种主要方法。

Prime+Probe: 探测方先用自身数据填满目标缓存组 (Prime)，等待被攻击方执行，再重新访问自身数据、测量时间 (Probe)。若被攻击方访问了同一缓存组，驱逐了探测方的数据，Probe 阶段即可发现缓存未命中。Prime+Probe 无需共享内存、也无需 CLFLUSH，适用面更广。局限在于精度低 (组级别分辨率，而非缓存行级别)，噪声也更大——探测方自身的数据可能被其他进程驱逐。Liu 等人在

跨 VM 场景下验证了 Prime+Probe 攻击 LLC 的可行性。

Evict+Time: 探测方先通过大量访存将目标数据从缓存中驱逐 (Evict)，再触发被攻击方加密、测量总执行时间 (Time)。被驱逐的数据若在加密中被重新加载，总执行时间将增大。Evict+Time 无需共享内存，但精度最差：仅能测量整次加密的总时间，无法定位具体哪个缓存行被访问。此外 Evict+Time 需要能够触发被攻击方的加密操作。

三种方法的比较如表2-1所示。

表 2-1 三种缓存侧信道攻击方法比较

特性	Flush+Reload	Prime+Probe	Evict+Time
共享内存要求	需要	不需要	不需要
CLFLUSH 要求	需要	不需要	不需要
空间分辨率	缓存行	缓存组	整体
噪声水平	低	中	高
跨核心能力	有	有	有限
隐蔽性	较低	中	较高

2.6 缓解措施概述

针对 Flush+Reload 这类缓存侧信道攻击，研究者提出了多种缓解思路，大致可分为软件和硬件两个层面。王崇等人对缓存侧信道防御技术做了系统梳理，从攻击模型角度对现有防御措施做了分类比较。

2.6.1 软件层面缓解

缓存刷新 (Cache Flushing): 执行敏感操作后，主动用 CLFLUSH 刷新相关缓存行，清除缓存状态中的信息残留。实现简单，性能开销却很明显——单次 CLFLUSH 的延迟约 40–200 周期，每次加密后刷新所有 T 表缓存行 (64 个) 的总开销在 2500–10000 周期级别。仅刷新 4 个监控缓存行 (每张 T 表的第 0 号缓存行) 时，性能下降约 5.7%；若刷新全部 64 个缓存行，开销将更为显著。此外还有一个问题：缓存刷新只能清除操作之后的痕迹，操作中的信息泄露无法阻止。

噪声注入 (Noise Injection): 在敏感操作前后插入随机延迟或随机访存，干扰时间测量。随机延迟是在加密前后引入 $\text{rand}() \bmod T_{\max}$ 周期的空循环。随机访存是以一定概率读取 T 表监控缓存行来引入假阳性噪声。本实验设置了三种噪声强度：低噪声 (30% 访问概率)、中噪声 (40% 访问概率)、高噪声 (50% 访问概率)。噪声注入的性能代价取决于强度：低噪声对性能影响较小，但防护效

果有限；高噪声能明显降低攻击成功率，但性能损失也更大。噪声注入的局限是概率性的——探测方可以通过累积样本量来消除噪声影响。

核心绑定 (Core Pinning)：`sched_setaffinity` 将敏感进程绑定在特定核心上，减少跨核心缓存共享的机会。但核心绑定只能阻止监听方利用共享 L3 进行跨核攻击，同核攻击无法阻止（双方运行于同一核心的超线程上）。多租户云环境中，核心绑定还面临资源分配的难题：为每个安全域独占核心，资源利用率将大幅下降。

缓存无关实现 (Cache-Oblivious Implementation)：设计不依赖缓存状态的算法实现，如 AES 的位切片方案。位切片将多个数据块打包到 SIMD 寄存器中并行处理，不使用查表操作，从源头上切断了缓存侧信道攻击的条件。局限在于位切片需同时处理多个数据块（通常为 8 块甚至更多），单块加密无法使用；实现复杂度也远高于 T 表方案。

2.6.2 硬件层面缓解

Intel CAT (Cache Allocation Technology)：允许 OS 将 LLC 的缓存路划分为多个分区，不同安全域的进程使用不同分区，实现缓存隔离。例如，12 路组相联的 LLC，0-5 路划给安全域 A，6-11 路划给安全域 B，两个域的缓存访问互不干扰。CAT 的优势在于硬件级隔离，性能代价低。局限也明显：分区粒度粗（以路为单位），可用路数有限，分区数量增多时每个分区的缓存容量不足。Wang 和 Lee 较早提出了基于缓存分区的防御思路，Intel CAT 可视为该思路的工程落地。

STEALTHMEM：Kim 等人研究的 STEALTHMEM 系统，在 OS 层面为敏感数据分配专用缓存行，其他进程无法使用这些缓存行。具体方法是修改页面着色策略，确保敏感页面映射到专用缓存组，其他进程的页面不会映射到同一组。实验表明 STEALTHMEM 能有效抵御 Prime+Probe 和 Flush+Reload 攻击。缺点是需要修改 OS 内核，部署成本高。

ARM 缓存分区与内存标签：ARM 架构提供了多种硬件隔离机制。ARMv8-A 的缓存分区特性（如 MPAM, Memory System Resource Partitioning and Monitoring）允许系统软件将缓存路划分给不同安全域，实现类似 Intel CAT 的缓存隔离效果。ARMv8.5-A 引入的内存标签扩展（MTE）为内存访问添加标签检查，能检测越界访问，间接增加侧信道攻击的复杂度。此外，ARMv8.3-A 的指针认证代码（PAC）主要用于防御控制流劫持，与缓存侧信道无直接关系，但其密码学认证机制可作为纵深防御的一环。

随机化缓存架构：Wang 和 Lee 提出的随机化缓存架构 (New Cache Designs)，在缓存索引计算中引入密钥相关的随机化，使探测方无法预知目标数据映射到哪

个缓存组。该方案需要修改处理器微架构，部署周期长，但从根本上破坏了缓存攻击的地址可预测性前提。

现有的缓解措施各有局限性，尚无单一方案能够消除缓存侧信道风险。实际部署需组合多种措施，在安全性和性能之间寻求平衡。张伟娟等人对缓存侧信道攻防最新进展做了系统总结，第四章将对几种软件层面的缓解措施进行量化评估，比较防护效果和性能开销。

2.7 本章小结

本章聚焦从硬件到防御的原理，阐述了 Flush+Reload 探测所需的技术背景。

硬件层面覆盖了 CPU 缓存的分层结构、寻址方式、替换策略和包含性关系，以及 MESI 一致性协议使跨核攻击成为可能的原因。共享内存部分介绍了 POSIX API、写时复制机制、虚拟化环境中 KSM 内存去重带来的安全风险。测量层面讨论了 RDTSC 和 RDTSCP 的差异、用户态测量面临的各种噪声来源、中位数阈值方法的原理。算法层面从 AES 的有限域数学基础出发，阐述了四个轮函数、密钥扩展的递归规律，T 表加速实现及其缓存侧信道漏洞。攻击层面展开了 Flush+Reload 的四步循环、Flush+Flush 变体，并与 Prime+Probe 和 Evict+Time 进行了对比。防御层面概述了缓存刷新、噪声注入、核心绑定、缓存无关实现等软件方案，以及 CAT、STEALTHMEM、随机化缓存等硬件方案的基本思路和适用场景。

下一章将给出具体搭建的实验环境和实现方案。

第三章 攻击方法设计与实现

本章给出 AES Flush+Reload 攻击测试框架的完整设计与实现方案。攻击框架分两个阶段协同工作：第一阶段通过 Flush+Reload 侧信道监控 T 表缓存行的访问模式，利用排除法恢复最后一轮子密钥；第二阶段复用同一批监控数据，结合 AES 等效解密的代数推导恢复倒数第二轮子密钥，经密钥扩展逆运算反解出原始密钥。针对 AES-128、AES-192 和 AES-256 三种密钥长度的差异，框架采用统一的 T 表监控策略和差异化的密钥恢复流程：AES-128 仅需第一阶段即可完成攻击；AES-192 和 AES-256 则需要两个阶段联合工作。在方法论层面，本章详细阐述排除法的数学原理——从单样本分析到多样本统计收敛的完整推导链，以及等效块解密如何将缓存访问信息转化为倒数第二轮密钥约束条件。在系统实现层面，本章描述基于父子进程模型的系统架构、进程间同步通信协议、量化评估指标体系的设计与实现，以及 CPU 核心绑定、随机噪声注入和缓存刷新等缓解措施的工程实现方案。

3.1 第一阶段：恢复最后一轮子密钥

3.1.1 AES 最后一轮特征

AES 加密的最后一轮没有 MixColumns。设最后一轮输入状态为 P ，密钥为 K_{last} ，输出密文为 C ：

$$C = ShiftRows(SubBytes(P)) \oplus K_{last} \quad (3-1)$$

ShiftRows 实现行位移，但 T 表可以将 ShiftRows 融合到查表映射中（详见下文）。设 S 为 S 盒， P_i 表示与密文字节 C_i 对应的 SubBytes 输入字节（即经 ShiftRows 重排后产生 C_i 的那个输入字节）， $K_{last,i}$ 为最后一轮密钥的第 i 个字节：

$$C_i = S(P_i) \oplus K_{last,i} \quad (3-2)$$

3.1.2 T 表结构与监控策略

T 表加速用 4 张预计算表 Te_0, Te_1, Te_2, Te_3 。每张 Te 表 256 项 $\times$ 4 字节 = 1024 字节 = 16 个缓存行。索引 0-15 占第 0 号缓存行，16-31 占第 1 号，以此类

推。攻击方仅监听每张 T 表的第 0 号缓存行，共 4 个。该信息量足以排除错误候选，同时可降低 `clflush` 开销和测量噪声。

仅监控第 0 缓存行的充分性分析，可以从两个角度：信息量和效率。

从信息量上看，每监测到一次未命中，就可以排除 64 个候选值，也就是排除一次 1/16 的搜索空间。为什么呢？以 Te_0 为例：在 Flush+Reload 攻击中，如果 Te_0 的缓存行未被访问，则意味着本次加密中所有使用 Te_0 的输入字节都不在 0~15 范围内，而 Te_0 负责的是第 2 行的输出，因此会影响第 2 行的 4 个密钥字节（即字节位置 2, 6, 10, 14）。同理： Te_1 未命中会影响第 3 行（字节 3, 7, 11, 15）， Te_2 未命中会影响第 0 行（字节 0, 4, 8, 12）， Te_3 未命中会影响第 1 行（字节 1, 5, 9, 13）。

需要特别说明，上述 Te_k 与状态行的对应关系是针对最后一轮的，与标准轮中 Te_k 对应第 k 行的规则不同。原因在于：本实验的受害者实现在最后一轮复用 Te 表并通过掩码提取 S 盒字节（详见 3.1.3 节），而每个 Te 表的 S 盒值位于 32 位条目的不同字节位置—— Te_0 的 S 盒值在字节 1 和 2， Te_2 的 S 盒值在字节 0 和 3。最后一轮从 Te_2 提取字节 0（最高有效字节）作为第 0 行输出，从 Te_3 提取字节 1 作为第 1 行输出，从 Te_0 提取字节 2 作为第 2 行输出，从 Te_1 提取字节 3（最低有效字节）作为第 3 行输出。因此最后一轮的映射为 $Te_0 \rightarrow$ 第 2 行、 $Te_1 \rightarrow$ 第 3 行、 $Te_2 \rightarrow$ 第 0 行、 $Te_3 \rightarrow$ 第 1 行，恰好是标准轮映射的循环移位。排除法针对的是最后一轮密钥恢复，因此使用最后一轮的映射关系。第二阶段恢复倒数第二轮密钥时，目标轮为标准轮，映射回到 $Te_k \rightarrow$ 第 k 行（详见 3.2.2 节）。

因此若 Te_0 的缓存行未被访问就会影响第 2 行的 4 个密钥字节，对于给定的密文字节 C_i ，索引落在 0 15 对应的密钥候选值为 $C_i \oplus Sbox[l]$ ($l = 0 \dots 15$)，恰好 16 个候选。由于索引不可能为 0 15，这 16 个候选值可在本次样本中被“排除”（即排除计数 +1）。一次未命中的总排除量就等于 4 个字节 $\times$ 16 个候选 = 64 个候选值。

如果监控全部 16 行会怎样？每次加密中 4 张 T 表需被访问多次，实际未被访问的缓存行数量较少。以具体数值为例：AES-128 中 Te_0 在 10 轮中共被访问 40 次，每次独立以 1/16 概率落在任意给定缓存行。单个缓存行未被访问的概率 $(15/16)^{40} \approx 0.076$ ，期望未命中缓存行数约 $16 \times 0.076 \approx 1.2$ 。监控全部 16 行平均可获取约 1.2 个未命中缓存行的排除信息，而仅监控第 0 行平均可获取约 0.076 个。全监控的信息量约为单行监控的 16 倍，但监控成本亦为 16 倍——每单位成本的信息效率几乎相同。

从效率上看，监控 k 个缓存行意味着每次 Flush+Reload 需执行 k 次 `clflush` 和 k 次内存访问测量。`clflush` 是串行化指令，延迟约 100-200 个 CPU 周期。监

控 4 行（每张 T 表 1 行）的总刷新开销约 400-800 周期。监控 64 行（每张 T 表 16 行）则需 6400-12800 周期。后者不仅显著增大每次采样的时间成本，测量窗口亦随之扩大，更多噪声和抢占干扰将被引入。因此仅监控第 0 缓存行是信息量与测量效率之间的最优平衡点。

3.1.3 最后一轮的 T 表访问方式与攻击前提

标准 AES 最后一轮不含 MixColumns，主流的高性能软件实现（如 OpenSSL）通常在最后一轮使用一个独立的、仅包含 S 盒替换值的查找表（例如一个 256 字节的 S 盒），而非复用 Te 表——因为最后一轮没有 MixColumns 操作，使用 Te 表再提取 S 盒字节是多余的。但在本实验中的受害者实现并非如此：最后一轮仍访问 Te 表，通过位掩码仅提取每个 32 位条目中对应纯 S 盒值的那个字节。

该实现细节对 Flush+Reload 攻击是有利的：最后一轮虽无 MixColumns，但仍会在 Te 表上留下缓存访问痕迹。监听方监控 Te 表即可捕获最后一轮的查表行为。而且由于最终仅使用 S 盒字节，密文字节与查表索引的关系不变：

$$C_i = S(P_i) \oplus K_{last,i} \quad (3-3)$$

需要明确的是，本文的攻击框架针对的是一种在最后一轮仍复用 Te 表的 AES 实现。然而，攻击方法学本身是通用的：无论受害者最后一轮用 Te 表还是独立的 S 表，攻击者监控的目标内存区域和排除法的数学原理都同样适用，只需调整监控的查找表地址即可。如果实现改用独立 S 盒（比如 OpenSSL），监听方将监控目标从 Te 表缓存行替换为 S 盒缓存行，排除公式依然成立。

3.1.4 T 表查表索引与密钥的代数关系

要理解排除法为何能恢复密钥，需要从 T 表查表的代数结构出发，阐明密钥、明文和 T 表索引之间的推导链。

标准轮的 T 表查表过程。AES 标准轮（非最后一轮）对状态矩阵的每一列执行 SubBytes、ShiftRows 和 MixColumns 三个操作。以第 r 轮的第 j 列为例，设输入状态字节为 $p_{0,j}$, $p_{1,j}$, $p_{2,j}$, $p_{3,j}$ ，轮密钥字节为 $k_{0,j}$, $k_{1,j}$, $k_{2,j}$, $k_{3,j}$ ，T 表查表过程为：

$$\begin{pmatrix} s'_{0,j} \\ s'_{1,j} \\ s'_{2,j} \\ s'_{3,j} \end{pmatrix} = Te_0[p_{0,j}] \oplus Te_1[p_{1,j+1}] \oplus Te_2[p_{2,j+2}] \oplus Te_3[p_{3,j+3}] \oplus \begin{pmatrix} k_{0,j} \\ k_{1,j} \\ k_{2,j} \\ k_{3,j} \end{pmatrix} \quad (3-4)$$

其中 $p_{i,j}$ 的下标 j 经过 ShiftRows 变换, 列号偏移量分别为 0、1、2、3。每个 Te_i 的查表索引就是对应的状态字节 $p_{i,j+i}$ 。

最后一轮的 T 表查表索引。如上节所述, 受害者在最后一轮访问 Te 表并通过掩码提取 S 盒字节, 因此查表索引就是状态字节 P_i 。设最后一轮输入为 P , 密文为 C , 最后一轮密钥为 K_{last} :

$$C_i = S(P_i) \oplus K_{last,i} \quad (3-5)$$

反解出最后一轮的查表索引:

$$P_i = S^{-1}(C_i \oplus K_{last,i}) \quad (3-6)$$

攻击者已知密文 C_i , 如果猜测到密钥候选 k , 即可计算对应的查表索引 $S^{-1}(C_i \oplus k)$ 。该索引落在哪个缓存行, 完全由索引的高 4 位决定:

$$CacheLine = \lfloor S^{-1}(C_i \oplus k) / 16 \rfloor \quad (3-7)$$

这是因为 T 表的结构与缓存行的映射关系所决定的。每个 T 表有 256 个条目 (索引 0x00~0xFF), 每个条目占 4 字节, 总大小为 1024 字节。缓存行大小为 64 字节, 每个缓存行可存放 $64/4 = 16$ 个连续的 T 表条目, 因此第 0 个缓存行存放索引 0~15, 第 1 个缓存行存放索引 16~31, 依此类推。一个 8 位的索引值 (0~255) 中, 低 4 位 (bit0~bit3) 决定该条目在缓存行内的偏移位置, 高 4 位 (bit4~bit7) 决定该条目属于第几个缓存行。因此缓存行号等于索引右移 4 位, 即 $\lfloor \text{索引} / 16 \rfloor$ 。在攻击中, 攻击者无法直接获知完整的 8 位索引, 但通过 Flush+Reload 观察到某个缓存行是否被访问, 就能推断索引的高 4 位信息。例如, 若第 0 号缓存行未被访问, 则索引不可能是 0~15, 从而排除 16 个候选值——这正是排除法能够逐步收敛密钥的根本原因。

为方便后续表述, 将上面两个公式写为:

$$\text{index}_k = S^{-1}(C_i \oplus k) \quad (3-8)$$

$$H(\text{index}_k) = \lfloor S^{-1}(C_i \oplus k) / 16 \rfloor \quad (3-9)$$

通过 Flush+Reload 观测该 T 表第 0 缓存行的访问状态 (被访问即命中, 未被访问即未命中)。若第 0 缓存行未被访问, 则实际加密过程中没有任何索引落在该行, 所有 $H(S^{-1}(C_i \oplus k)) = 0$ 的候选 k 与观测矛盾, 予以排除; 若被访问, 则无法确定具体是哪个索引触发了访问, 不提供排除信息。

排除条件仅在未命中时生效：

$$\text{排除候选} \Leftrightarrow \text{该 T 表第 0 缓存行未被访问} \wedge H(S^{-1}(C_i \oplus k)) = 0 \quad (3-10)$$

3.1.5 排除法的数学推导

在多样本累积与统计收敛中，为每个密钥字节初始化一个排除计数器数组，256 个候选初始全 0。

对于 N 个样本，第 j 个样本的密文字节是 $C_i^{(j)}$ 。 $m^{(j)}$ 表示第 j 次加密中该 T 表第 0 缓存行是否没被访问（ $m^{(j)} = 1$ 是未命中， $m^{(j)} = 0$ 是命中）。排除计数 $CK[i][k]$ 定义为密钥字节 i 的候选 k 被排除的总次数：

$$CK[i][k] = \sum_{j=1}^N m^{(j)} \cdot \mathbf{1}[H(S^{-1}(C_i^{(j)} \oplus k)) = 0] \quad (3-11)$$

$\mathbf{1}[\cdot]$ 是指示函数，条件成立取 1 否则取 0。命中时 $m^{(j)} = 0$ ，整项为 0，不计数。未命中时 $m^{(j)} = 1$ ，若 $H(S^{-1}(C_i^{(j)} \oplus k)) = 0$ 则计数加 1。设 k^* 为密钥字节 i 的真实值，正确候选 k^* 始终不会被排除。其原因在于：正确索引 $S^{-1}(C_i^{(j)} \oplus k^*)$ 对应的缓存行一定被访问过，未命中时 $H(S^{-1}(C_i^{(j)} \oplus k^*)) \neq 0$ ，因此 $CK[i][k^*] = 0$ 。错误候选的 $CK[i][k] > 0$ 。计数最少的候选就是要恢复的密钥字节。

期望样本量分析。排除法仅在未命中样本上执行排除，命中样本直接略过。设总样本量 N ，单次加密中某 T 表第 0 缓存行不被访问的概率记作 p_{miss} 。以下推导基于查表索引均匀分布的假设——在攻击者已知密文、密钥未知且服从均匀分布的条件下，该假设基本成立。单次 T 表查表索引落在第 0 缓存行的概率是 1/16，不落的是 15/16。

由于每次加密中同一 Te 表被多个字节多次访问（每轮 4 次，4 列各 1 次），AES-128 共有 9 轮标准轮（含 MixColumns）加 1 轮最终轮（无 MixColumns），本实现最终轮仍复用 Te 表，故每张 Te 表被访问次数为：标准轮 $9 \times 4 = 36$ 次，最终轮 4 次，合计 40 次；AES-192 为 11 标准轮加 1 最终轮，每表 $11 \times 4 + 4 = 48$ 次；AES-256 为 13 标准轮加 1 最终轮，每表 $13 \times 4 + 4 = 56$ 次。在均匀随机假设下，单次访问不落在第 0 缓存行的概率为 15/16，因此 $p_{miss} = (15/16)^n$ ，其中 n 为该 Te 表的总访问次数。代入得 AES-128 的 $p_{miss} \approx 0.076$ ，AES-192 的 $p_{miss} \approx 0.045$ ，AES-256 的 $p_{miss} \approx 0.027$ 。期望有效样本数约为 $N \cdot p_{miss}$ 。需要注意，上述 n 的取值以受害者实现在所有轮次（包括最后一轮）均访问 Te 表为前提；若最后一轮改用独立 S 盒表，则 Te 表的总访问次数减少 4 次（最后一轮不再查 Te 表）， p_{miss} 将略有增大，但推导方法不变。

每个有效样本对单个密钥字节可排除 $16/256 = 1/16$ 的候选。对某个错误候选 k ，单次样本将其排除的概率为 $p_{miss}/16$ （需同时满足未命中且 $H(S^{-1}(C_i \oplus k)) = 0$ ），保留概率为 $1 - p_{miss}/16$ 。初始候选集 256 个，经过 N 个样本后期望剩余的候选数：

$$E[\text{remaining}] = 256 \cdot \left(1 - \frac{p_{miss}}{16}\right)^N \quad (3-12)$$

要让剩余候选数等于 1，代入上述 p_{miss} 估计，由 $E[\text{remaining}] = 256 \cdot (1 - p_{miss}/16)^N = 1$ 解得 $N \approx \ln(1/256)/\ln(1 - p_{miss}/16)$ ，当 p_{miss} 较小时 $\ln(1 - p_{miss}/16) \approx -p_{miss}/16$ ，线性近似为 $N \approx 16 \ln(256)/p_{miss} \approx 89/p_{miss}$ 。AES-128 需总样本量约 $89/0.076 \approx 1171$ ，AES-192 约 $89/0.045 \approx 1978$ ，AES-256 约 $89/0.027 \approx 3296$ 。

需要强调，上述 $89/p_{miss}$ 仅是单字节独立恢复的近似下界，不能直接作为整体攻击的样本量推荐。整体攻击需要 16 个密钥字节同时恢复，联合成功概率远低于单字节概率。假设单字节恢复成功率为 p_{byte} ，整体成功率约为 p_{byte}^{16} ；对于两阶段攻击（AES-192/256），整体成功率进一步约为 p_{byte}^{32} 。因此实际需要的样本量远超上述估算，原因包括多个层面：样本之间存在相关性、测量本身存在噪声、需要保留安全冗余，其中最重要的是 16 个字节需同时恢复，任意字节出错即导致整体失败，因此需要远多于单字节理论值的样本数以提升联合置信度。本实验里 AES-128、AES-192、AES-256 分别使用 5000、9000、15000 个样本。

经过大量样本累积后，理论上真实值 k^* 的排除计数始终为 0，而错误候选的排除计数逐渐增大。最终排除计数最少的候选即为恢复的密钥字节。

误判率分析。排除法的可靠性取决于两个条件：正确候选始终不被排除，错误候选被充分排除。实际测量中存在噪声，可能导致两种误判：

假阴性：正确候选 k^* 被错误排除。根源在于 Te 表第 0 缓存行实际上被访问了，但监听方测到了未命中。Flush 和 Reload 之间的时间窗口内，缓存行可能被其他进程驱逐、CPU 预取器换出，或者 OS 调度导致 Victim 尚未完成加密，监听方即开始 Reload。假阴性率记作 P_{FN} ，它使 $CK[i][k^*] > 0$ ，正确候选的排除计数不再为零。

假阳性：错误候选 k 未被排除。这发生在 Te 表第 0 缓存行实际上未被访问，但监听方测到了命中。原因有多种：其他进程（如共享库代码）恰好访问了同一缓存行、测量噪声使访问时间低于阈值。假阳性率记作 P_{FP} ，它使错误候选的排除计数偏低，收敛速度减慢。假阳性不直接导致错误恢复，但需累积更多样本方能将错误候选与正确候选区分开。

设单次测量的假阴性率 P_{FN} 、假阳性率 P_{FP} ，有效未命中概率修正为 $p'_{miss} =$

$p_{miss}(1 - P_{FP}) + (1 - p_{miss})P_{FN}$ 。低噪声环境 ($P_{FN} < 0.01$, $P_{FP} < 0.01$) 下, 修正量对收敛速度的影响可以忽略。高噪声环境则需按比例增加样本量予以补偿。本实验在可控环境中进行, 噪声水平低, 假阴性和假阳性率均控制在 1% 以下。

3.1.6 AES-128 密钥恢复

AES-128 恢复最后一轮密钥 K_{10} 以后, 通过密钥扩展的逆过程即可获得原始密钥 K_0 。AES-128 密钥扩展生成 44 个 32 位字 $w_0, w_1, \dots, w_{43}$, K_{10} 对应 $w_{40}, w_{41}, w_{42}, w_{43}$ 。

密钥扩展逆过程:

$$w_{i-4} = \begin{cases} w_i \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/4] & \text{if } i \bmod 4 = 0 \\ w_i \oplus w_{i-1} & \text{otherwise} \end{cases} \quad (3-13)$$

其中 $\text{Rcon}[r]$ 为 AES 第 r 轮的轮常量, SubWord 对 32 位字的每个字节做 S 盒替换, RotWord 将 32 位字循环左移一个字节。

从 w_{43} 逐步逆向推到 w_0, w_1, w_2, w_3 , 即可获得原始密钥 K_0 。

3.2 第二阶段: 恢复倒数第二轮子密钥

AES-192 和 AES-256 仅恢复最后一轮子密钥无法反推原始密钥, 还需获取倒数第二轮子密钥。攻击者使用同一批 Flush+Reload 监控数据, 结合已知的最后一轮密钥和密文, 计算解密中间值, 进而恢复倒数第二轮密钥。

3.2.1 复用监控信息与等效块解密

第一阶段 Flush+Reload 仅记录了整次加密中各 T 表缓存行是否被访问, 无法区分各轮次的访问。攻击者无法从中提取倒数第二轮独有的访问特征。然而从另一角度考虑: 缓存行未被访问, 即意味着所有轮次均未访问该行。攻击者可将该未命中信息应用于倒数第二轮的中间状态, 通过排除法恢复倒数第二轮密钥。

还有一个隐含前提: 受害者 AES 实现在所有轮次中访问的是同一组 Te 表 (标准轮使用完整的 32 位查表结果, 最后一轮通过掩码仅提取 S 盒字节)。这样才能保证 Flush+Reload 监控到的 Te 表缓存行未命中信息对所有轮次均有效。

等效块解密方法的推导思路。第二阶段的核心方法为等效块解密, 攻击者无需执行完整 AES 解密, 仅需计算密文到倒数第二轮加密输入状态 S_{r-1} 的映射即可。 S_{r-1} 的每个字节, 恰好就是第 $r-1$ 轮查 Te 表时的输入索引。

推导的关键在于理解 AES 等效解密结构。标准 AES 解密依次执行 In-

vShiftRows、InvSubBytes、AddRoundKey、InvMixColumns。但该顺序与加密过程的逆序并非直接对应：加密中 SubBytes 在 ShiftRows 之前，解密中 InvShiftRows 在 InvSubBytes 之前。为使解密结构更规整，NIST 标准定义了“等效解密”：将 InvSubBytes 和 InvShiftRows 交换位置（二者可交换，因为 InvSubBytes 逐字节操作，InvShiftRows 仅改变位置），然后将轮密钥做 InvMixColumns 变换，得到 $Kd_r = \text{InvMixColumns}(K_r)$ 。

在等效解密框架下，从密文 C 出发，经过一轮等效解密得到的中间值 D_2 恰好等于 $\text{ShiftRows}(\text{SubBytes}(S_{r-1}))$ 。这正是第 $r-1$ 轮加密中 SubBytes 和 ShiftRows 的输出。由此可以直接反推 S_{r-1} ，而 S_{r-1} 的每个字节就是第 $r-1$ 轮查 Te 表的输入索引。攻击者只需已知密文 C 和最后一轮密钥 K_{last} ，加上猜测的倒数第二轮密钥 K_{last-1} （或其等效形式 Kd_1 ），即可在本地完成所有计算。

AES-128 为何不需要第二阶段。AES-128 的密钥扩展生成 44 个 32 位字 $w_0, w_1, \dots, w_{43}$ ，原始密钥 K_0 对应 w_0, w_1, w_2, w_3 ，最后一轮密钥 K_{10} 对应 $w_{40}, w_{41}, w_{42}, w_{43}$ 。密钥扩展的递推关系为每 4 个字一组， w_i 仅依赖于 w_{i-4} 和 w_{i-1} 。从 $w_{40}, w_{41}, w_{42}, w_{43}$ 出发，可以逐步逆向推导出 $w_{36}, w_{32}, \dots, w_0$ ，完整恢复全部 44 个字。因此 AES-128 仅凭最后一轮密钥 K_{10} 即可唯一确定原始密钥 K_0 ，无需恢复倒数第二轮密钥。

AES-192 和 AES-256 则不同——仅凭最后一轮子密钥的 4 个字，无法逆向推出完整原始密钥。这是由每个 AES 变体的密钥长度 Nk 与每轮子密钥长度（固定 4 字）之间的不匹配造成的。密钥扩展逆过程需要 Nk 个连续字才能回溯，但最后一轮子密钥只提供 4 个字，当 $Nk > 4$ 时就会出现缺口。这就是 AES-192/256 必须额外恢复倒数第二轮子密钥的根本原因。下面结合逆推公式做具体说明。

AES-192 的密钥扩展。 $Nk = 6$ ，一共生成 52 个字 $w_0, \dots, w_{51}$ 。递推关系以 6 个字为一组：

$$w_i = \begin{cases} w_{i-6} \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/6] & i \bmod 6 = 0 \\ w_{i-6} \oplus w_{i-1} & \text{否则} \end{cases} \quad (3-14)$$

逆过程只需将等式移项：

$$w_{i-6} = \begin{cases} w_i \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/6] & i \bmod 6 = 0 \\ w_i \oplus w_{i-1} & \text{否则} \end{cases} \quad (3-15)$$

攻击者从 $K_{12} = (w_{48}, w_{49}, w_{50}, w_{51})$ 出发，共有 4 个已知字。逆推过程如图 3-1 所示：

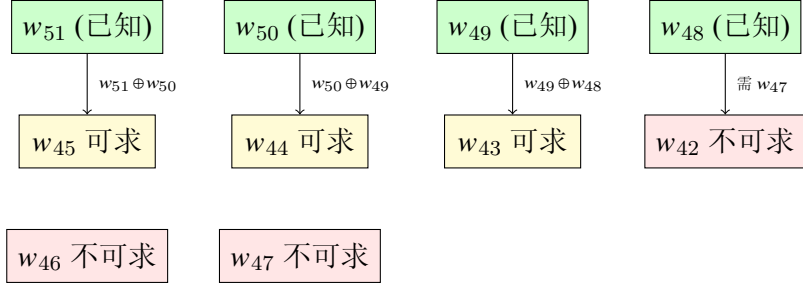


图 3-1 AES-192 从 K_{12} 的 4 个已知字逆向推导。 w_{45}, w_{44}, w_{43} 可直接异或求得； w_{42} 需要 w_{47} 但 w_{47} 未知，逆推中断。 w_{46}, w_{47} 也无法从已有信息获取

具体地：

- $w_{51} = w_{45} \oplus w_{50} \Rightarrow w_{45} = w_{51} \oplus w_{50}$ ，可求。
 - $w_{50} = w_{44} \oplus w_{49} \Rightarrow w_{44} = w_{50} \oplus w_{49}$ ，可求。
 - $w_{49} = w_{43} \oplus w_{48} \Rightarrow w_{43} = w_{49} \oplus w_{48}$ ，可求。
 - $w_{48} = w_{42} \oplus \text{SubWord}(\text{RotWord}(w_{47})) \oplus \text{Rcon}[8]$ ： w_{47} 未知， w_{42} 不可求。
- 到此逆推中断—— w_{42} 求不出， w_{46} 和 w_{47} 也无从获得，无法继续回到 $w_0 \sim w_5$ 。唯有同时恢复 $K_{11} = (w_{44}, w_{45}, w_{46}, w_{47})$ ，凑齐 $w_{44} \sim w_{51}$ 共 8 个连续字，方可沿逆过程一路回推到 w_0 。

AES-256 的密钥扩展。 $Nk = 8$ ，一共生成 60 个字 $w_0, \dots, w_{59}$ 。AES-256 多了 $i \bmod 8 = 4$ 这一特例（需经过 *SubWord* 而非裸异或）。递推关系：

$$w_i = \begin{cases} w_{i-8} \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/8] & i \bmod 8 = 0 \\ w_{i-8} \oplus \text{SubWord}(w_{i-1}) & i \bmod 8 = 4 \\ w_{i-8} \oplus w_{i-1} & \text{否则} \end{cases} \quad (3-16)$$

逆过程：

$$w_{i-8} = \begin{cases} w_i \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/8] & i \bmod 8 = 0 \\ w_i \oplus \text{SubWord}(w_{i-1}) & i \bmod 8 = 4 \\ w_i \oplus w_{i-1} & \text{否则} \end{cases} \quad (3-17)$$

攻击者从 $K_{14} = (w_{56}, w_{57}, w_{58}, w_{59})$ 出发：

- $w_{59} = w_{51} \oplus w_{58} \Rightarrow w_{51}$ 可求。
- $w_{58} = w_{50} \oplus w_{57} \Rightarrow w_{50}$ 可求。
- $w_{57} = w_{49} \oplus w_{56} \Rightarrow w_{49}$ 可求。
- w_{56} 需要 w_{48} 和 w_{55} ， w_{55} 未知， w_{48} 不可求。

同理， $w_{52}, w_{53}, w_{54}, w_{55}$ 均无法从 4 个已知字中推出，逆推在 w_{48} 处中断。必须同时恢复 $K_{13} = (w_{52}, w_{53}, w_{54}, w_{55})$ ，获得 $w_{52} \sim w_{59}$ 共 8 个连续字才能完整逆推。

小结。AES-128 的 $Nk = 4$ ，一轮子密钥的 4 个字恰好等于 Nk ，逆推通路。AES-192 的 $Nk = 6$ 、AES-256 的 $Nk = 8$ ， $Nk > 4$ 导致一轮子密钥信息不足，必须额外恢复倒数第二轮子密钥，补齐 Nk 个连续字才能打通逆推链路。这正是两阶段攻击的数学根源。

3.2.2 从密文到 S_{13} 的推导与缓存行映射

以 AES-256 为例。攻击者在第一阶段获得了密文 C 和最后一轮子密钥 K_{14} ，需要从密文推导出倒数第二轮加密的输入状态 S_{13} ，并建立 S_{13} 到 Flush+Reload 可观测缓存行的映射关系。

第一步：密文与最后一轮密钥异或。

$$D_{1,i} = C_i \oplus K_{14,i}, \quad i = 0, 1, \dots, 15 \quad (3-18)$$

攻击者已知 C 和 K_{14} ， D_1 的每个字节均可精确计算。

第二步：通过逆 T 表计算中间值 IV_1 。AES 解密使用 4 张逆 T 表 Td_0, Td_1, Td_2, Td_3 ，每张将逆 S 盒替换与 InvMixColumns 系数融合为单次查表，结构与加密 T 表 Te 对称。 IV_1 的每个字节由 4 次逆 T 表查表异或得到：

$$\begin{aligned} IV_{1,i} = & Td_{\sigma(i,0)}[D_1[\pi(i,0)]] \oplus Td_{\sigma(i,1)}[D_1[\pi(i,1)]] \\ & \oplus Td_{\sigma(i,2)}[D_1[\pi(i,2)]] \oplus Td_{\sigma(i,3)}[D_1[\pi(i,3)]] \end{aligned} \quad (3-19)$$

其中 $\sigma(i, \cdot)$ 和 $\pi(i, \cdot)$ 分别由 InvShiftRows 和逆列混合的矩阵结构决定。以第 0 字节为例：

$$IV_{1,0} = Td_0[D_1[0]] \oplus Td_1[D_1[13]] \oplus Td_2[D_1[10]] \oplus Td_3[D_1[7]] \quad (3-20)$$

此处 Td_k 的下标对应 InvShiftRows 后第 k 行的字节。需要指出，部分实现（如将 InvShiftRows 吸收到 Td 表定义内部的方案）中 Td_1 和 Td_3 的系数行互换，此时 D_1 的索引变为对称形式，功能上完全等价。本文沿用 Td_k 下标与 InvShiftRows 后行号一致的约定。

第三步：异或等效解密密钥得到 D_2 。

$$D_{2,i} = IV_{1,i} \oplus Kd_{1,i} \quad (3-21)$$

其中等效解密轮密钥 $Kd_1 = \text{InvMixColumns}(K_{13})$ ，即对 K_{13} 的每一列施加 In-

vMixColumns 矩阵变换：

$$\begin{pmatrix} Kd_{1,0,j} \\ Kd_{1,1,j} \\ Kd_{1,2,j} \\ Kd_{1,3,j} \end{pmatrix} = \begin{pmatrix} 0x0E & 0x0B & 0x0D & 0x09 \\ 0x09 & 0x0E & 0x0B & 0x0D \\ 0x0D & 0x09 & 0x0E & 0x0B \\ 0x0B & 0x0D & 0x09 & 0x0E \end{pmatrix} \begin{pmatrix} K_{13,0,j} \\ K_{13,1,j} \\ K_{13,2,j} \\ K_{13,3,j} \end{pmatrix} \quad (3-22)$$

第四步：从 D_2 反推 S_{13} 。加密第 13 轮依次为 $SubBytes \rightarrow ShiftRows \rightarrow MixColumns \rightarrow AddRoundKey(K_{13})$ 。由等效解密的结构关系：

$$D_2 = ShiftRows(SubBytes(S_{13})) \quad (3-23)$$

逐字节反推：

$$S_{13,i} = S^{-1}(D_{2,\rho(i)}) \quad (3-24)$$

其中 $\rho(i)$ 为 InvShiftRows 的字节重排映射。 S_{13} 的每个字节就是第 13 轮加密时查 Te 表的输入索引。

第五步： S_{13} 到 T 表缓存行的映射。第 13 轮为标准轮，状态矩阵按行划分，每行字节查同一张 Te 表：

$$S_{13,i} \rightarrow Te_{i \bmod 4} \quad (3-25)$$

即第 0 行字节 ($i = 0, 4, 8, 12$) 查 Te_0 ，第 1 行字节 ($i = 1, 5, 9, 13$) 查 Te_1 ，第 2 行字节 ($i = 2, 6, 10, 14$) 查 Te_2 ，第 3 行字节 ($i = 3, 7, 11, 15$) 查 Te_3 。索引的高 4 位决定缓存行：

$$H(S_{13,i}) = S_{13,i} \gg 4 \quad (3-26)$$

攻击者无法直接观测 $S_{13,i}$ ，但可通过 Flush+Reload 的缓存行未命中信息间接排除：若 $Te_{i \bmod 4}$ 的第 0 缓存行在整次加密中全局未被访问，则第 13 轮中 $H(S_{13,i}) \neq 0$ 。

3.2.3 复用监控数据直接恢复 Kd_1

第一阶段 Flush+Reload 记录了加密中每次对 Te 表的缓存访问。某个 Te 表的第 0 缓存行，如果在整次加密中均未被访问过，则不仅是最后一轮，所有轮次（包括第 13 轮）均未访问该行。

由 3.2.2 节的推导， $Kd_{1,i} = IV_{1,i} \oplus D_{2,i}$ ，而 $D_{2,i} = S(S_{13,\rho^{-1}(i)})$ （经 ShiftRows 重排后的 S 盒输出）。因此，枚举 S_{13} 的候选值等价于枚举 Kd_1 的候选值。本实现采用直接恢复 Kd_1 的策略：对 Kd_1 的每个字节枚举 256 个候选值，利用缓存行未命中信息排除不可能的候选，无需先恢复 S_{13} 再推导 Kd_1 。

以 Kd_1 的某个字节为例说明排除过程。该字节位于第 row 行、第 col 列，对应状态矩阵中位置 $i = row + 4 \cdot col$ 。在第 j 次加密中，攻击者已知 $IV_{1,i}^{(j)}$ 和该次

Flush+Reload 的观测结果。关键推导分三步：

从缓存观测推导 $S_{13,i}$ 的约束。 $S_{13,i}$ 是第 13 轮查 Te_{row} 表的输入索引。若 Te_{row} 的第 0 缓存行在本次加密中全局未命中，则所有轮次（包括第 13 轮）均未访问该缓存行，即第 row 行所有字节的查表索引均不落在 0~15 范围内。因此 $S_{13,i} \neq l$ ($l = 0, \dots, 15$)。

从 $S_{13,i}$ 的约束推导 $D_{2,\rho(i)}$ 的约束。 $D_{2,\rho(i)} = S(S_{13,i})$ (S 盒替换，索引由 ShiftRows 重排)。由 $S_{13,i} \neq l$ 得出 $D_{2,\rho(i)} \neq \text{Sbox}[l]$ ($l = 0, \dots, 15$)。

从 $D_{2,\rho(i)}$ 的约束推导 $Kd_{1,i}$ 的排除。由 3.2.2 节的推导， $Kd_{1,i} = IV_{1,i} \oplus D_{2,\rho(i)}$ 。因此 $Kd_{1,i}$ 不能等于 $IV_{1,i} \oplus \text{Sbox}[l]$ ，这 16 个候选值可在本次样本中被排除。

定义 Kd_1 的排除计数 $CKd_1[i][k]$ 为第 i 字节的候选值 k 被排除的累积次数：

$$CKd_1[i][k] = \sum_{j=1}^N m_{row}^{(j)} \cdot \mathbf{1}[k \in \{IV_{1,i}^{(j)} \oplus \text{Sbox}[l] \mid l = 0, \dots, 15\}] \quad (3-27)$$

其中 $m_{row}^{(j)}$ 表示第 j 次加密中 Te_{row} 的第 0 缓存行是否未被访问（1 为未命中，0 为命中）， $\mathbf{1}[\cdot]$ 为指示函数。正确候选的排除计数趋近于零，经过足够多样本累积后，排除计数最少的候选即为恢复的 $Kd_{1,i}$ 值。

上述直接恢复 Kd_1 的方法与“先恢复 S_{13} 再推导 Kd_1 ”在数学上完全等价：排除 $S_{13,i}$ 的候选索引 s （满足 $H(s) = 0$ ），等价于排除 Kd_1 候选 $IV_{1,i} \oplus \text{Sbox}[s]$ 。直接法省去了中间步骤，实现更简洁。

3.2.4 从 Kd_1 计算倒数第二轮子密钥

Kd_1 恢复出来后，对其每一列做 MixColumns 变换即可得到加密倒数第二轮子密钥 K_{13} (AES-256) 或 K_{11} (AES-192)：

写成矩阵形式，对 Kd_1 的每一列做 MixColumns：

$$\begin{pmatrix} K_{13,0,j} \\ K_{13,1,j} \\ K_{13,2,j} \\ K_{13,3,j} \end{pmatrix} = \begin{pmatrix} 0x02 & 0x03 & 0x01 & 0x01 \\ 0x01 & 0x02 & 0x03 & 0x01 \\ 0x01 & 0x01 & 0x02 & 0x03 \\ 0x03 & 0x01 & 0x01 & 0x02 \end{pmatrix} \begin{pmatrix} Kd_{1,0,j} \\ Kd_{1,1,j} \\ Kd_{1,2,j} \\ Kd_{1,3,j} \end{pmatrix} \quad (3-28)$$

3.2.5 从轮密钥恢复原始密钥

AES-192 恢复 K_{12} 和 K_{11} 后获得连续 8 个 32 位字 ($w_{44}-w_{51}$)，足以完整逆推 192 位原始密钥 K_0 ；AES-256 恢复 K_{14} 和 K_{13} 后获得连续 8 个 32 位字 ($w_{52}-w_{59}$)，同样可完整逆推 256 位原始密钥 K_0 。

AES-192 逆向推导：

$$w_{i-6} = \begin{cases} w_i \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/6] & \text{if } i \bmod 6 = 0 \\ w_i \oplus w_{i-1} & \text{otherwise} \end{cases} \quad (3-29)$$

AES-256 逆向推导:

$$w_{i-8} = \begin{cases} w_i \oplus \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{Rcon}[i/8] & \text{if } i \bmod 8 = 0 \\ w_i \oplus \text{SubWord}(w_{i-1}) & \text{if } i \bmod 8 = 4 \\ w_i \oplus w_{i-1} & \text{otherwise} \end{cases} \quad (3-30)$$

3.2.6 两阶段攻击流程总结

AES-128: 使用 5000 样本, 排除法恢复 K_{10} , 逆过程恢复 K_0 。

AES-192: 使用 9000 样本, 阶段一恢复 K_{12} , 阶段二复用监控数据直接恢复 Kd_1 , 经 MixColumns 计算 K_{11} , 逆向恢复 K_0 。

AES-256: 使用 15000 样本, 阶段一恢复 K_{14} , 阶段二复用监控数据直接恢复 Kd_1 , 经 MixColumns 计算 K_{13} , 逆向恢复 K_0 。

关键点在于: 两个阶段共享同一批监控数据。攻击者只需执行一遍 Flush+Reload, 即可同时恢复两个轮密钥。

以 AES-256 为例, 整个密钥恢复的数学过程可概括为以下公式链。设 $C^{(j)}$ 为第 j 次加密的密文, $m_k^{(j)} \in \{0, 1\}$ 表示 Te_k 的第 0 缓存行是否未被访问。

阶段一: 恢复最后一轮子密钥 K_{14} 。对每个密钥字节 i ($i = 0, \dots, 15$), 利用最后一轮映射关系 (3.1.3 节), 若对应 Te 表的第 0 缓存行未命中, 则排除满足 $H(S^{-1}(C_i^{(j)} \oplus k)) = 0$ 的候选 k :

$$CK[i][k] = \sum_{j=1}^N m_*^{(j)} \cdot \mathbf{1}[H(S^{-1}(C_i^{(j)} \oplus k)) = 0] \quad (3-31)$$

排除计数最少的候选即为恢复的 $K_{14,i}$ 。

阶段二: 直接恢复等效解密密钥 Kd_1 。对 Kd_1 的每个字节 i ($i = 0, \dots, 15$), 利用标准轮映射 (i 所在行 row 对应 Te_{row}), 若 Te_{row} 的第 0 缓存行未命中, 则排除 16 个候选值 $IV_{1,i}^{(j)} \oplus \text{Sbox}[l]$ ($l = 0, \dots, 15$):

$$CKd_1[i][k] = \sum_{j=1}^N m_{row}^{(j)} \cdot \mathbf{1}[k \in \{IV_{1,i}^{(j)} \oplus \text{Sbox}[l] \mid l = 0, \dots, 15\}] \quad (3-32)$$

密钥恢复链。

$$Kd_1 \xrightarrow{\text{MixColumns}} K_{13}, \quad \{K_{13}, K_{14}\} \xrightarrow{\text{密钥扩展逆过程}} K_0 \quad (3-33)$$

AES-192 的结构与此完全一致，仅将 K_{14} 换为 K_{12} 、 K_{13} 换为 K_{11} 、 Kd_1 中的 K_{13} 换为 K_{11} 。AES-128 则只需执行阶段一，由 K_{10} 直接逆推 K_0 。

3.3 系统架构设计

系统按模块划分为六个部分。共享内存模块创建和管理共享内存，放置 T 表数据。Victim 进程模块模拟受害者，执行 AES 加密。Attacker 进程模块执行 Flush+Reload，收集缓存访问信息。密钥恢复模块从收集的信息中恢复 AES 密钥。量化评估模块计算各项指标，评估攻击效果。自动化实验模块支持批量测试和结果可视化。

系统采用父子进程模型，通过 fork 创建 Victim 进程。父进程为 Attacker，负责刷新缓存、收集样本、恢复密钥。子进程为 Victim，执行 AES 加密响应 Attacker 的请求。进程间通过管道同步通信，确保攻击流程的时序正确。

3.3.1 进程间通信协议

Attacker 和 Victim 之间通过两对管道（pipe）实现同步，分别用于请求-响应方向。管道 1（Attacker→Victim）传递加密请求信号，管道 2（Victim→Attacker）传递加密完成通知信号。

单次采样的通信过程是：Attacker 先执行 Flush 阶段，用 clflush 刷新目标 T 表缓存行；然后通过管道 1 向 Victim 发送 1 字节的请求信号（'\E' 表示"Encrypt"）；Victim 从管道 1 读取请求信号后随即执行一次 AES 加密；加密完成后，Victim 将 16 字节密文写入共享内存的指定字段，然后通过管道 2 向 Attacker 发送 1 字节的响应信号（'\D' 表示"Done"）；Attacker 从管道 2 读取响应信号后，立即执行 Reload 阶段，测量目标缓存行的访问时间；然后从共享内存中读取密文，记录本次采样的缓存命中/未命中状态和密文，进入下一轮采样。密文通过共享内存传递而非管道，避免了管道传输 16 字节数据的额外延迟。

管道通信采用阻塞读写模式，天然实现了进程间的同步：Attacker 在发送请求后阻塞等待 Victim 的响应，Victim 在处理完前一个请求前不会接收新请求。这种设计避免了忙等待（busy-wait）带来的 CPU 资源浪费和时序干扰。管道缓冲区大小为 64KB（Linux 默认值），远大于单次通信的数据量，不会发生写阻塞。

3.4 量化评估指标体系

3.4.1 基础指标

样本总数 $sample_count$ 、测量次数 $measurement_count$ (样本数 $\times 4$)、命中次数 hit_count 、未命中次数 $miss_count$ 、命中率 hit_rate (命中次数 $\div$ 总测量次数)、未命中率 $miss_rate$ 。

样本总数反映攻击的数据需求, 测量次数将样本数按监控目标数 (4 张 T 表) 展开, 反映总测量开销。命中率和未命中率直接决定排除法的收敛速度: 未命中率越高, 有效排除样本越多, 收敛越快。在理想无噪声环境下, 未命中率的理论值可由前文中的 p_{miss} 公式预测, 实际测量值与理论值的偏差反映了噪声水平。

3.4.2 时间统计指标

基本统计量 (均值、标准差、最小值、最大值) 和分位数指标 (中位数、Q1、Q3、P5、P95)。本实验使用分位数而不使用均值标准差, 因为分位数对异常值的抵抗力强得多, 更适合用户态高精度时间测量的场景。更进一步讲, 用户态时间测量受操作系统调度、中断和动态频率调节的影响, 异常值不可避免。均值和标准差对异常值敏感, 少量极端值即可大幅偏移均值。分位数 (特别是中位数和四分位距) 对异常值具有鲁棒性, 能更准确地反映典型访问时间的分布特征。P5 和 P95 则提供了分布尾部的信息, 有助于区分缓存命中和未命中的时间阈值选取。时间统计指标与基础指标之间存在关联: 当命中率异常偏高时, 访问时间的分布会向低值偏移 (因为缓存命中时间远低于未命中时间), 此时 P95 可以作为判断噪声水平的辅助指标。

3.4.3 泄露带宽

泄露带宽 (Leakage Bandwidth) 定义为单位时间内侧信道攻击从目标进程中提取的信息量, 单位为比特/秒 (bps)。它综合了两个因素: 每次观测所获取的信息量 (互信息) 和单位时间内的观测次数 (采样频率)。泄露带宽越高, 攻击者在单位时间内获取的密钥信息越多, 攻击速度越快。

$$BW = 4 \cdot MI_{binary} \cdot f_{sample} \quad (3-34)$$

该公式的含义是: 一次完整采样对 4 个缓存行 (Te_0 、 Te_1 、 Te_2 、 Te_3 各 1 个) 各进行一次观测, 每次观测提供 MI_{binary} 比特信息, 故一次采样共获取 $4 \cdot MI_{binary}$ 比特 (假设各观测独立); 每秒完成 f_{sample} 次采样, 两者相乘即为每秒获取的总

信息量。因此，提升单次观测的互信息（降低噪声）或提升采样频率（缩短单次采样耗时）都能增大泄露带宽，加速密钥恢复。

互信息 互信息用于量化单次 Flush+Reload 观测所获取的信息量。下面分步解释其含义与计算方法。

第一步：理解观测过程。每次 Flush+Reload 观测，攻击者执行以下操作：先用 `clflush` 将目标缓存行逐出缓存，然后等待 Victim 执行一次 AES 加密，最后测量该缓存行的访问时间。如果访问时间短（命中），说明 Victim 访问了该缓存行；如果访问时间长（未命中），说明 Victim 未访问。因此，单次观测的结果是一个二值信息：命中或未命中。

第二步：建立信息模型。将上述观测过程建模为二值信道。信道的输入是 Victim 的真实访问状态：设 $X = 1$ 表示 Victim 访问了目标缓存行， $X = 0$ 表示未访问。信道的输出是攻击者的观测结果：设 $Y = 1$ 表示命中， $Y = 0$ 表示未命中。攻击者的目标是通过观测 Y 来推断 X ，因为 X 与密钥相关——Victim 访问哪个 T 表缓存行取决于密钥字节值，观测到“未命中”可以排除一个错误密钥候选。

第三步：确定信道参数。信道的特性由条件概率 $P(Y|X)$ 描述。当 Victim 访问了目标缓存行（ $X = 1$ ）时，攻击者必然观测到命中（ $Y = 1$ ），即 $P(Y = 1|X = 1) = 1$ 。当 Victim 未访问（ $X = 0$ ）时，若无噪声则攻击者应观测到未命中，但噪声注入会以概率 f 产生假命中，即 $P(Y = 1|X = 0) = f$ （无噪声时 $f = 0$ ）。输入 X 的先验概率由命中率决定： $P(X = 1) = p_{access}$ ， $P(X = 0) = 1 - p_{access}$ 。

第四步：计算互信息。互信息 $I(X; Y)$ 衡量观测 Y 对推断 X 的贡献。从直观理解：在观测之前，攻击者对 X 的不确定性是 $H(X)$ 比特（ X 的熵）；观测到 Y 之后，剩余的不确定性是 $H(X|Y)$ 比特（条件熵）。观测消除的不确定性即为获得的信息量： $I(X; Y) = H(X) - H(X|Y)$ 。将二值信道的概率参数代入，可推导出互信息的计算公式：

$$MI_{binary} = \sum_{x \in \{0,1\}} \sum_{y \in \{0,1\}} P(X = x, Y = y) \cdot \log_2 \frac{P(X = x, Y = y)}{P(X = x) \cdot P(Y = y)} \quad (3-35)$$

其中联合概率 $P(X = x, Y = y)$ 由 $P(X = x) \cdot P(Y = y|X = x)$ 计算得到，边缘概率 $P(Y = y)$ 由 $\sum_x P(X = x, Y = y)$ 计算得到。

第五步：理解结果。当 $f = 0$ （无噪声）时，观测结果 Y 完全确定输入 X （命中对应访问，未命中对应未访问），此时互信息达到最大值 $MI_{binary} = H(p_{access})$ ，即每次观测提供 $H(p_{access})$ 比特信息。当 $f > 0$ 时，噪声使部分观测结果不可靠，互信息下降。当缓存刷新启用时，攻击者始终观测到未命中，观测结果与输入无

关，互信息为零。

采样频率 在本项目中，一次采样指的是攻击者完成一轮完整的 Flush+Reload 测量流程：用 clflush 刷新 4 张 T 表第 0 号缓存行 → 通知 Victim 加密一次 → 等待 Victim 完成 → Reload 测量 4 个缓存行的访问时间 → 记录命中/未命中状态和密文。因此 f_{sample} 就是单位时间内能完成上述完整轮次的次数：

$$f_{sample} = \frac{f_{CPU}}{C_{avg}} \quad (3-36)$$

其中 f_{CPU} 为 CPU 主频 (Hz)， C_{avg} 为单次采样的平均 CPU 周期数。 C_{avg} 通过实验测量：记录整个采样过程的起止时间戳 (rdtsc)，用总周期数除以样本数即得。以实验平台 (i5-1135G7, 2.4GHz) 为例，AES-128 基准配置下 C_{avg} 约 17500 周期， f_{sample} 约 137000 次/秒。密钥越长，单次采样涉及的加密和等待开销越大， C_{avg} 越高， f_{sample} 越低，泄露带宽也随之下降。

3.4.4 攻击成功率

$$Success_Rate = \frac{N_{success}}{N_{total}} \times 100\% \quad (3-37)$$

其中 N_{total} 为总实验轮数， $N_{success}$ 为成功恢复完整密钥的轮数。

攻击成功率是最终的综合评估指标，受上述所有指标的间接影响。样本总数决定了排除法的收敛程度，未命中率决定了有效信息量，时间统计指标反映了测量质量，泄露带宽量化了信息泄露的速率。在实际评估中，攻击成功率与样本数呈单调递增关系：样本数不足时成功率接近 0，超过阈值后迅速上升至接近 100%。该阈值受噪声水平和密钥长度的影响，噪声越高、密钥越长，所需样本数越多。泄露带宽则从另一个维度评估攻击威胁：即使攻击成功率相同，泄露带宽越高意味着攻击者在单位时间内能获取更多信息，攻击速度越快。因此，泄露带宽和攻击成功率从速率和可靠性两个互补的角度刻画了侧信道攻击的威胁程度。

3.5 缓解措施实现

本实验实现了三种针对 Flush+Reload 攻击的配置措施——CPU 核心绑定、随机噪声注入和缓存刷新，可单独使用也可组合。

3.5.1 CPU 核心绑定

实验中 Attacker 和 Victim 通过 `sched_setaffinity` 绑定到同一 CPU 核心。同核运行时两个进程交替使用同一套 L1d/L2 缓存，缓存状态的变化直接反映受害者的访问行为，测量信号最清晰。由于 x86 架构的 `clflush` 指令会从所有缓存层级驱逐指定缓存行，核心绑定并非一种独立的缓解措施，本节仅将其作为实验的基本配置。

3.5.2 随机噪声注入

随机噪声注入的方法为：Victim 加密完成后随机访问 T 表监控缓存行，干扰攻击者的观测。攻击者依赖“缓存行未被访问 = 未命中”的推理，如果 Victim 自身在加密以外额外访问了监控缓存行，攻击者的未命中观测即不再可靠。

噪声注入采用的是访问式注入（access-based）：Victim 加密完成后，以概率 p_{noise} 对每张 T 表的第 0 号缓存行执行一次内存读取操作（`mov` 指令），将该缓存行重新加载到 CPU 缓存中。低噪声 $p_{noise} = 30\%$ ，中噪声 $p_{noise} = 40\%$ ，高噪声 $p_{noise} = 50\%$ 。读取操作使用普通的内存加载指令（`mov`），不使用 `clflush`，读取目标为第 0 号缓存行内的随机表项（索引 0-15）。

噪声注入对排除法的影响如下。当第 0 缓存行在加密中未被访问（真实未命中）时，噪声以概率 p_{noise} 将其重新加载到缓存中，使攻击者观测到命中（假阳性）。因此有效未命中率变为：

$$p'_{miss} = p_{miss} \times (1 - p_{noise}) \quad (3-38)$$

有效排除样本数从 $N \cdot p_{miss}$ 减少为 $N \cdot p_{miss} \times (1 - p_{noise})$ 。以 AES-128 为例，无噪声时 $p_{miss} \approx 0.076$ ，有效排除样本约 $5000 \times 0.076 = 380$ ；低噪声（ $p_{noise} = 30\%$ ）时 $p'_{miss} \approx 0.053$ ，有效排除样本约 266；高噪声（ $p_{noise} = 50\%$ ）时 $p'_{miss} \approx 0.038$ ，有效排除样本约 190。噪声越强，有效排除样本越少，收敛速度越慢，需要更多样本才能达到相同的密钥恢复置信度。

3.5.3 缓存刷新

缓存刷新是最有效的缓解措施：Victim 加密完成后立即用 `clflush` 指令驱逐 T 表监控缓存行。实现方式为对 4 张 Te 表的第 0 号缓存行（即攻击者监控的目标缓存行）执行两次 `clflush` 指令，两次之间插入延迟和内存屏障（`mfence`、`lfence`），确保缓存行被彻底驱逐。

缓存刷新使攻击者的 Reload 阶段始终观测到缓存未命中，无论 Victim 是

否访问过 T 表。此时命中率降为 0，攻击者无法区分“Victim 访问了该缓存行”和“Victim 未访问该缓存行”两种情况，Flush+Reload 攻击完全失效。

缓存刷新的性能开销：8 次 clflush（4 个缓存行 $\times$ 2 次）加 2 次 mfence、2 次 lfence，外加 100 次空循环延迟，按每次 clflush 约 100 周期、每次 mfence 和 lfence 约 30 周期估算，clflush 与屏障开销约 920 周期，加上延迟循环，总开销约 1000 周期，约为一次 AES-128 加密（约 200 周期）的 5 倍。虽然开销远小于刷新全部 128 个缓存行的方案，但在对性能敏感的场景中仍不可忽视。缓存刷新适合作为安全测试的基准对照（验证攻击在最强防御下确实失效），而非实际部署的缓解方案。

3.6 本章小结

本章给出了 AES Flush+Reload 攻击测试框架的设计与实现。

攻击方法的核心是排除法。第一阶段通过监控 T 表第 0 缓存行的访问状态，利用密文与查表索引的代数关系逐步排除错误密钥候选，恢复最后一轮子密钥。本文进行了完整推导：单样本分析、多样本累积统计、期望样本量估计。仅监控第 0 缓存行的策略在信息量与测量效率之间实现了最优平衡。

第二阶段采用等效块解密方法，将同一批 Flush+Reload 监控数据复用于倒数第二轮密钥恢复。从密文出发，通过等效解密推导出解密中间值 IV_1 ，然后对 Kd_1 的每个字节直接执行排除法——枚举 S 盒输出值，计算对应的 Kd_1 候选并排除。恢复 Kd_1 后，经 MixColumns 变换得到加密倒数第二轮子密钥。AES-128 仅凭最后一轮密钥即可逆推原始密钥，无需第二阶段。AES-192 和 AES-256 必须恢复倒数第二轮密钥才能完整逆推原始密钥。

系统实现层面，父子进程模型和管道同步通信协议保证了攻击流程的时序正确，量化评估指标体系涵盖基础指标、时间统计指标、泄露带宽和攻击成功率，综合衡量了攻击的数据需求、测量质量、信号强度和泄露速率，缓解措施实现了随机噪声注入和缓存刷新，其中缓存刷新能使攻击完全失效，但性能开销亦较大。

第四章 实验与分析

本章在 Intel Core i5-1135G7 平台上对前述攻击框架进行系统性的实验验证和量化评估。实验涵盖 AES-128、AES-192 和 AES-256 三种密钥长度变体，每种变体分别测试基准配置（无缓解措施）和四种缓解配置（低/中/高强度随机噪声注入、缓存刷新），每轮配置运行 30 次完整攻击以获得统计可靠的评估结果。实验内容分为三个层面：首先通过基准测试验证攻击框架的正确性和有效性，测量缓存命中/未命中的时间分布并校准分类阈值；其次对比不同缓解配置下的攻击成功率和泄露带宽，量化各防御措施的实际防护效果；最后深入分析泄露带宽与成功率之间的非线性关系、多阶段攻击的级联误差放大效应等关键现象。本章的实验数据将为第三章设计的攻击方法和缓解措施提供实证支撑，同时揭示 Flush+Reload 侧信道攻击在实际系统环境中的威胁程度和防御可行性。

4.1 实验环境

4.1.1 硬件平台

实验所用处理器为 Intel Core i5-1135G7（Tiger Lake 微架构），该处理器拥有 4 个物理核心、8 个逻辑线程（支持超线程），基础频率 2.40GHz，最大睿频 4.20GHz。其缓存层次结构参数见表 4-1。

表 4-1 实验平台处理器缓存参数。L1d 和 L2 为各核私有，L3 为所有核心共享。缓存行大小为 64 字节

缓存层级	容量	相联度	写策略	共享范围
L1d	48KB	12 路	写回	单核私有
L2	1.25MB	20 路	写回	单核私有
L3	8MB	12 路	写回	全核共享

L3 为全核共享，即使攻击者和受害者运行于不同的逻辑核心上，攻击者也能用 `clflush` 指令将目标缓存行从整个缓存层次中驱逐，然后通过 `reload` 的延迟判断受害者是否再次访问了该行。

64 字节的缓存行大小意味着 T 表每个表项（256 个 4 字节 `uint32_t` 元素，恰好占 1KB）跨越 16 个缓存行。攻击者可以逐行监控，获取细粒度的访问模式。

T 表总大小 ($4 \times 1\text{KB} = 4\text{KB}$) 远小于 L3 容量, 不会因缓存容量不足而被意外驱逐。这为攻击提供了稳定的缓存驻留条件。

4.1.2 软件环境

操作系统为 Ubuntu 20.04 LTS, Linux 内核版本 5.15.0。编译器采用 GCC 9.4.0, 编译选项为 `-O0 -march=native`。-O0 不开启优化, 保证 T 表访问不被编译器优化掉, 这对 Flush+Reload 攻击至关重要。-march=native 使编译器针对 Tiger Lake 微架构生成特定指令序列, 包括对 `rdtsc`、`clflush` 等指令的支持。

内核配置方面, 实验中关闭了地址空间布局随机化 (ASLR), 以排除 ASLR 对共享库内存映射的影响。同时禁用了 CPU 频率动态调节 (通过将 `scaling_governor` 设为 `performance` 模式), 确保 CPU 始终运行在标称频率, 避免因频率波动导致的时间测量偏差。此外, 实验保留了超线程功能, Attacker 和 Victim 均通过 `sched_setaffinity` 绑定到同一逻辑核心 (CPU 0), 两个进程在该核心上交替执行, 共享同一套 L1d/L2 缓存, 确保缓存状态的变化直接反映受害者的访问行为。

4.1.3 实验参数

实验参数设置如下: AES-128、AES-192、AES-256 的样本量分别为 5000、9000、15000, 每种配置运行 30 轮。

测试了 5 种缓解配置: Baseline (无缓解措施)、Noise-Low (30% 访问概率)、Noise-Medium (40% 访问概率)、Noise-High (50% 访问概率)、Cache-Flush (加密后刷新 T 表监控缓存行)。

4.2 基准测试结果

4.2.1 攻击成功率

AES-128/192/256 各运行 30 轮完整成功率测试, 在基准测试下 (无缓解措施), 三种变体的成功率均为 100% (30/30)。在受控实验环境中, 缓存命中与未命中时间表现出显著的、无重叠的双峰分布, 这使得分类器能以极低的误判率工作。当样本量足以满足排除法所需的统计置信度时, 密钥恢复成功率达到 100% 是符合理论预期的。

实验样本量 (分别为 5000、9000、15000 次加密) 远超理论最低要求。对于 AES-128 的单字节攻击, 由 $89/p_{miss}$ 估算的理论最低样本量约为 1171 次, 而实际需要恢复 16 个字节且保证联合置信度, 所需样本量更高。实验采用的 5000 个样本为单字节恢复提供了充足的统计冗余, 确保了高成功率。对于 AES-192 和

AES-256 的两阶段攻击，更大的样本量（9000 和 15000）同样提供了充足的统计冗余，使得基准成功率达到 100%。

4.2.2 缓存时间分布与阈值校准

缓存命中/未命中的时间分布统计见表 4-2。

表 4-2 缓存访问时间分布统计（校准阶段测量）。命中时间中位数（118 周期）与未命中时间中位数（347 周期）相差约 229 周期，P5–P95 范围内无重叠，说明基于中位数的阈值设置能可靠地区分命中和未命中这两种状态

统计量	命中时间（周期）	未命中时间（周期）
中位数	118.0	347.0
第一四分位数（Q1）	116.0	344.0
第三四分位数（Q3）	120.0	362.0
第 5 百分位数（P5）	108.0	333.0
第 95 百分位数（P95）	132.0	508.0

阈值校准过程如下：在攻击开始前，攻击者对目标缓存行执行 10000 次 `clflush and reload` 测量，记录每次 `reload` 的 CPU 周期数。将测量结果按升序排列，分别取命中时间和未命中时间的中位数 M_{hit} 和 M_{miss} ，取两者中点作为分类阈值 T ：

$$T = \frac{M_{\text{hit}} + M_{\text{miss}}}{2} \quad (4-1)$$

在本实验中， $T = (118 + 347)/2 = 233$ 周期。`reload` 时间低于 T 判定为命中，高于 T 判定为未命中。

4.3 缓解措施对比测试

4.3.1 噪声缓解措施机制

噪声缓解的思路是：加密之后按一定概率干扰 T 表缓存行，打乱攻击者看到的缓存状态。Victim 完成 AES 加密后，以预设概率对 4 个 T 表的第 0 号缓存行执行内存读取操作（`mov` 指令），将该缓存行重新加载到 CPU 缓存中。这样，即使 Victim 加密时未访问某个 T 表的第 0 号缓存行（攻击者本应观察到 `cache miss`），噪声读取也会让攻击者看到 `cache hit`，从而引入假阳性，降低排除法密钥恢复的准确性。需要指出，本实验的噪声注入仅针对每张 T 表的第 0 号缓存行，即与攻击者监控目标相同的缓存行。这是一种针对性的噪声策略：由于攻击者只监控第 0 号缓存行，噪声也只需干扰该行的观测即可。

4.3.2 AES-128 缓解措施对比

AES-128 的结果见表 4-3。

表 4-3 AES-128 缓解措施对比测试结果（30 轮，样本量 5000）。噪声越强，成功率越低，泄露带宽也越低。Cache-Flush 下泄露带宽归零

配置	成功率	泄露带宽 (bps)
Baseline	100%	60759
Noise-Low (30%)	100%	26079
Noise-Med (40%)	90%	21873
Noise-High (50%)	46%	12467
Cache-Flush	0%	0

AES-128 的攻击仅需恢复 16 字节密钥，采用单阶段攻击即可完成。

在 Noise-Low（30%）配置下，AES-128 的成功率仍为 100%，说明低噪声概率对单阶段攻击影响有限；在 Noise-Medium（40%）配置下，成功率降至 90%；在 Noise-High（50%）配置下，成功率降至 46%，50% 的噪声概率对单阶段攻击产生显著影响。

关于泄露带宽：AES-128 在 Noise-High 下泄露带宽约 12kbps，从基准的约 61kbps 降至约 12kbps，降幅达 79%。泄露带宽随噪声增强而下降，成功率也相应下降，说明噪声缓解对单阶段攻击有明显效果。

4.3.3 AES-192 缓解措施对比

AES-192 的结果见表 4-4。

表 4-4 AES-192 缓解措施对比测试结果（30 轮，样本量 9000）。AES-192 要走两阶段攻击，对噪声更敏感

配置	成功率	泄露带宽 (bps)
Baseline	100%	34678
Noise-Low (30%)	93%	18080
Noise-Med (40%)	66%	12433
Noise-High (50%)	36%	9174
Cache-Flush	0%	0

AES-192 的密钥长度为 24 字节，攻击分两阶段：第一阶段恢复最后一轮子密钥 K_{12} （16 字节），第二阶段恢复倒数第二轮子密钥 K_{11} （16 字节），两者结合通过密钥扩展逆运算恢复原始 24 字节密钥。两阶段攻击对噪声更敏感，因为

第二阶段的正确性依赖第一阶段的结果。第一阶段恢复的 K_{12} 一旦有任意字节出错，第二阶段的等效解密计算就会全部错误，导致 K_{11} 恢复失败。

两阶段攻击的整体成功率是两个阶段成功率的乘积，噪声对每个阶段的削弱效应会级联放大。

对比 AES-128 和 AES-192 在相同噪声配置下的表现：在 Noise-Low（30%）下，AES-192 成功率降至 93%，而 AES-128 仍为 100%，两阶段攻击级联效应开始显现；在 Noise-Medium（40%）下，AES-192 成功率为 66%，而 AES-128 为 90%；在 Noise-High（50%）下，AES-192 成功率降至 36%，远低于 AES-128 的 46%。这个现象说明 AES-192 的两阶段攻击结构在噪声下级联效应显著，高噪声下成功率下降幅度明显大于单阶段攻击。

4.3.4 AES-256 缓解措施对比

AES-256 的结果见表 4-5。

表 4-5 AES-256 缓解措施对比测试结果（30 轮，样本量 15000）。AES-256 对噪声最敏感

配置	成功率	泄露带宽 (bps)
Baseline	100%	23584
Noise-Low (30%)	66%	10758
Noise-Med (40%)	40%	8329
Noise-High (50%)	20%	6032
Cache-Flush	0%	0

AES-256 对噪声最为敏感。在 Baseline 配置下成功率为 100%；在 Noise-Low（30%）配置下成功率降至 66%；在 Noise-Medium（40%）配置下成功率降至 40%；在 Noise-High（50%）下成功率降至 20%。

一个自然的问题是：AES-256 和 AES-192 的攻击结构完全相同——都是两阶段攻击，每个阶段恢复 16 字节轮密钥——为什么 AES-256 在各噪声配置下的成功率均显著低于 AES-192？答案不在于阶段数的差异，而在于每阶段能获取的有效信息量不同。AES-256 因加密轮数更多，导致攻击中“有效排除样本”更少，单字节恢复的可靠性更低；而两阶段攻击的级联放大效应将这一微小差异急剧放大，最终导致成功率显著下降。下面从三个层面展开分析。

（1）核心差异：每阶段的有效信息量。AES-192 和 AES-256 的攻击结构相同：第一阶段恢复最后一轮子密钥（AES-192 恢复 K_{12} ，AES-256 恢复 K_{14} ），第二阶段恢复倒数第二轮子密钥（AES-192 恢复 K_{11} ，AES-256 恢复 K_{13} ），然后通过密钥扩展逆运算得到原始密钥。关键区别在于加密轮数不同导致有效排除样本数

量不同。AES-192 有 12 轮加密，每个 T 表被访问 48 次；AES-256 有 14 轮加密，每个 T 表被访问 56 次。攻击依赖的是”某个缓存行在整次加密中从未被访问”这一事件，其发生概率为：

$$p_{miss} = \left(\frac{15}{16}\right)^{\text{访问次数}} \quad (4-2)$$

AES-192 的 $p_{miss} \approx 0.045$ ，AES-256 的 $p_{miss} \approx 0.027$ 。AES-256 的有效排除样本数量天生比 AES-192 少约 40%，单字节恢复的可靠性更低。

(2) 级联放大效应：微小差异被乘积急剧放大。两阶段攻击的整体成功率是阶段一和阶段二成功率的乘积，而阶段二的正确性完全依赖阶段一的结果——阶段一恢复的轮密钥一旦有任意字节出错，阶段二的等效解密计算就会全部错误。设阶段一恢复 16 字节密钥的成功概率为 $P_1 = p_{byte}^{16}$ ，阶段二在阶段一正确条件下的成功概率为 $P_{2|1} \approx p_{byte}^{16}$ ，则整体成功概率近似为：

$$P_{total} = P_1 \times P_{2|1} \approx p_{byte}^{32} \quad (4-3)$$

若 AES-192 的 $p_{byte} = 0.99$ ，则 $0.99^{32} \approx 0.72$ ；若 AES-256 的 $p_{byte} = 0.97$ ，则 $0.97^{32} \approx 0.38$ 。单字节可靠性仅差 2 个百分点，经过 32 次乘积后成功率几乎减半。实验数据也证实了这一点：在 Noise-Low 下 AES-192 成功率为 93% 而 AES-256 为 66%，在 Noise-Medium 下分别为 66% 和 40%，在 Noise-High 下分别为 36% 和 20%，差距随噪声增强而持续扩大。

(3) 噪声对 AES-256 打击更重。噪声注入进一步压缩有效未命中率：

$$p'_{miss} = p_{miss} \times (1 - p_{noise}) \quad (4-4)$$

以 Noise-High ($p_{noise} = 0.5$) 为例，AES-192 的有效未命中率从 0.045 降至 0.0225，AES-256 从 0.027 降至 0.0135。AES-256 的底层信息本来就少，再被噪声压缩，有效信息更加稀缺。而两阶段攻击中阶段二的输入完全依赖阶段一的输出——阶段一错一个字节，阶段二全错——这就是级联失败。

综合以上三个层面，AES-256 比 AES-192 更难攻破的根本原因可以概括为：两者攻击阶段数相同，但 AES-256 因加密轮数更多（14 轮 vs 12 轮）导致 p_{miss} 更低（0.027 vs 0.045），每阶段有效信息量少约 40%，单字节恢复的可靠性更低；而两阶段攻击需要同时恢复 32 个字节的轮密钥，32 次乘积的级联放大效应将微小的单字节可靠性差异急剧放大——单字节成功率仅差 2 个百分点，整体成功率几乎减半；噪声注入进一步压缩本已稀缺的有效信息，使级联失败更加严重。

再看样本量与成功率的关系。三种 AES 变体分别使用 5000、9000、15000 次样本，但在噪声环境下 AES-256 仍然最难攻破。这说明：对于多阶段攻击，仅依

靠增加样本量无法有效对抗噪声。每个阶段的错误会向后续阶段传播，更多样本仅能提升单个阶段的置信度，无法解决级联失败这一结构性问题。

4.3.5 三种 AES 缓解措施对比

三种 AES 在各缓解配置下的成功率，见图 4-1。

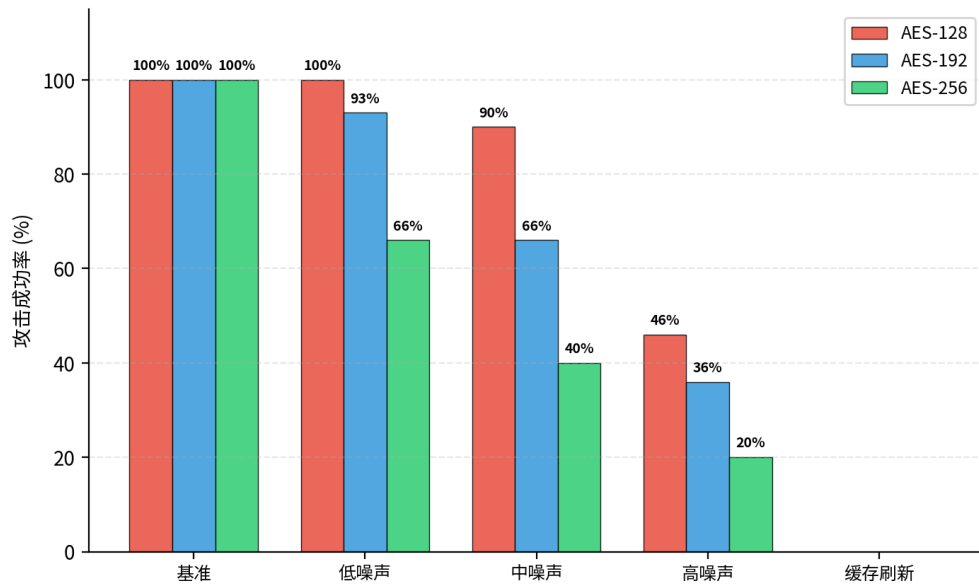


图 4-1 三种 AES 在不同缓解措施下的攻击成功率对比（30 轮）。Baseline 时三种 AES 成功率均为 100%。随着噪声提高，成功率会降低，密钥越长降低越快。Cache-Flush 使成功率降至零

图 4-1 直观地表明：三种 AES 变体的成功率随噪声强度增加而下降的速率差异显著。AES-128 在 Noise-Low 下成功率仍为 100%，Noise-Medium 下降至 90%，Noise-High 下降至 46%，单阶段攻击对噪声相对鲁棒；AES-192 在 Noise-Low 下为 93%，Noise-Medium 下降至 66%，Noise-High 下降至 36%，两阶段攻击级联效应显著；AES-256 在 Noise-Low 下为 66%，Noise-Medium 下为 40%，Noise-High 下为 20%，级联效应使噪声的影响被急剧放大。差异的根源在于：AES-256 加密轮数更多导致 p_{miss} 更低、每阶段有效信息量更少，而两阶段攻击的 32 字节乘积级联将微小的单字节可靠性差异急剧放大。

进一步分析另一现象：AES-256 在 Noise-Medium 下泄露带宽约 8kbps，成功率 40%；AES-192 在 Noise-Medium 下泄露带宽约 12kbps，成功率 66%。虽然 AES-256 的泄露带宽更低，但其成功率下降更剧烈。这说明噪声缓解并未完全消除侧信道的信息泄露，仅将泄露量降至攻击者难以有效利用的水平。从防御角度而言，这意味着仅依靠噪声可能不足。如果攻击者采用了更先进的信号处理或统计推断技术，仍可能从残余泄露中提取密钥。相比之下，Cache-Flush 使泄露带宽

降为零，防护更为彻底。

4.4 量化指标分析

4.4.1 泄露带宽分析

泄露带宽定义为单位时间内从缓存侧信道泄露的信息量（单位：bit/s），计算公式为：

$$BW = 4 \cdot MI_{binary} \cdot f_{sample} \quad (4-5)$$

其中 MI_{binary} 为单次缓存行观测的互信息（bit/观测）， f_{sample} 为采样频率（次/秒），系数 4 对应每次采样对 4 个缓存行各进行一次独立观测。

泄露带宽的下降幅度与 AES 变体和噪声强度直接相关。以基准泄露带宽为参照：

- AES-128: Noise-Low 下降 57% (60759→26079bps), Noise-Medium 下降 64% (→21873bps), Noise-High 下降 79% (→12467bps)
- AES-192: Noise-Low 下降 48% (34678→18080bps), Noise-Medium 下降 64% (→12433bps), Noise-High 下降 73% (→9174bps)
- AES-256: Noise-Low 下降 54% (23584→10758bps), Noise-Medium 下降 65% (→8329bps), Noise-High 下降 74% (→6032bps)

一个有趣的发现是：泄露带宽的下降幅度在三种 AES 变体间总体接近，但在低噪声级别下仍存在一定差异。Noise-Low 下三种变体的降幅分别为 57%、48%、54%，极差为 9 个百分点；而 Noise-Medium 和 Noise-High 下极差缩小至 1–6 个百分点。相比之下，成功率的下落差异在所有噪声级别下均十分显著。例如，Noise-High 下三种变体的泄露带宽降幅都在 73%–79% 之间，但 AES-128 成功率为 46%，AES-192 为 36%，AES-256 仅为 20%。这再次说明成功率下降的主因是 AES-256 每阶段有效信息量更少加之 32 字节乘积级联放大，而非单纯的泄露量减少。

Cache-Flush 下攻击者无法获得任何信息，泄露带宽归零。图 4-2 展示了泄露带宽的变化趋势。

4.4.2 泄露带宽与攻击成功率的关系

泄露带宽与攻击成功率之间存在非线性关系。AES-128 在 Noise-High 下泄露带宽约 12kbps，成功率 46%；AES-256 在 Noise-High 下泄露带宽约 6kbps，成功率仅 20%。两者泄露带宽相差较大，但成功率差异更为显著。这说明对于 AES-256，即使泄露量仍然较大，其更低的 p_{miss} 导致单字节可靠性更低，经过 32 字节乘积

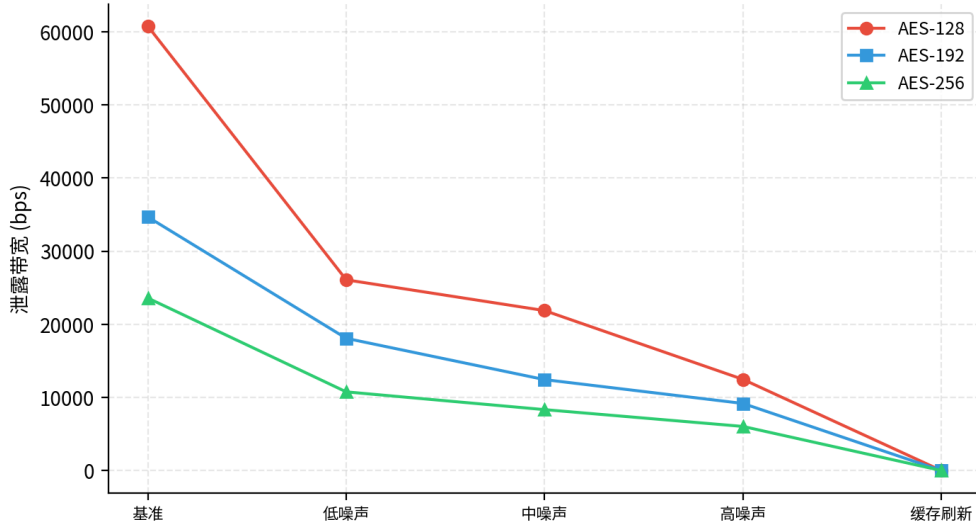


图 4-2 三种 AES 在不同缓解措施下的泄露带宽（30 轮）。Baseline: AES-128 约 61kbps, AES-192 约 35kbps, AES-256 约 24kbps。噪声越强，泄露带宽越低。Cache-Flush 下即为零

级联放大后成功率急剧下降。

4.5 本章小结

本章在 Intel Core i5-1135G7 平台上，对 AES-128/192/256 三种变体分别测试了基准配置和四种缓解配置，每种运行 30 轮。结论如下。

默认情况下三种 AES 变体成功率均为 100%。随着噪声强度的增加成功率呈下降趋势：AES-128 在 Noise-Low 下为 100%，Noise-Medium 下为 90%，Noise-High 下为 46%，单阶段攻击对噪声相对鲁棒；AES-192 在 Noise-Low 下为 93%，Noise-Medium 下为 66%，Noise-High 下为 36%，两阶段攻击级联效应显著；AES-256 在 Noise-Low 下为 66%，Noise-Medium 下为 40%，Noise-High 下为 20%。AES-256 与 AES-192 的攻击阶段数相同，但 AES-256 更难攻破的根本原因在于：14 轮加密使 p_{miss} 更低（0.027 vs 0.045），每阶段有效信息量少约 40%，单字节恢复可靠性更低；而两阶段攻击需要同时恢复 32 个字节的轮密钥，级联放大效应将微小的单字节可靠性差异急剧放大；噪声注入进一步压缩本已稀缺的有效信息，使级联失败更加严重。缓存刷新是确定性防御，使成功率与泄露带宽均降至零，防护最为彻底。

泄露带宽随噪声增强而下降，缓存刷新下归零。但泄露带宽与成功率之间并非线性关系：AES-128 在 Noise-High 下泄露带宽约 12kbps，成功率 46%；AES-256 在 Noise-High 下泄露带宽约 6kbps，成功率仅 20%。两者泄露带宽相差较大，但成功率差异更为显著，AES-256 更低的 p_{miss} 经 32 字节乘积级联放大后导致成功率急剧下降。

第五章 总结与展望

5.1 工作总结

本文围绕处理器微架构侧信道漏洞安全测试，使用 Flush+Reload 分析 AES 加密算法在多密钥长度下的攻击可行性和缓解措施的有效性。

在攻击框架上，本实验构建了一套支持 AES-128/192/256 三种密钥长度的统一测试框架，采用模块化设计。整个流程划分为六个功能模块：共享内存管理、Victim 进程模拟、Attacker 进程探测、密钥恢复、量化评估、自动化实验。Victim 和 Attacker 采用父子进程模型，管道同步通信确保攻击时序可控。共享内存模块使用 POSIX shm_open 和 mmap，T 表数据映射到双方地址空间，为 Flush+Reload 探测提供共享页面。自动化实验支持批量配置和结果可视化，不同参数组合的对比实验可高效完成。

密钥恢复策略为三种 AES 密钥长度分别做了适配。AES-128 通过排除法恢复最后一轮子密钥 K_{10} ，再通过密钥扩展逆过程推导原始密钥 K_0 。实验中 5000 个样本即可稳定恢复完整 128 位密钥。AES-192 和 AES-256 需采用两阶段：阶段一恢复最后一轮子密钥（AES-192 恢复 K_{12} ，AES-256 恢复 K_{14} ）。阶段二复用同一批 Flush+Reload 监控数据，结合等效解密中间值 IV_1 ，对等效解密轮密钥 Kd_1 的每个字节直接执行排除法，恢复 Kd_1 后经 MixColumns 变换得到加密倒数第二轮子密钥。复用监控数据的优势在于无需重复采样——AES-128、AES-192、AES-256 分别使用 5000、9000、15000 个样本。

评估体系涵盖基础指标、时间统计指标、泄露带宽和攻击成功率。基础指标包括样本总数、命中次数、未命中率。时间统计指标采用分位数（中位数、Q1、Q3、P5、P95）替代均值和标准差——分位数对用户态高精度时间测量中常见的异常值具有更强的鲁棒性。泄露带宽定义为 $BW = 4 \cdot MI_{binary} \cdot f_{sample}$ ，其中 MI_{binary} 为单次缓存行观测的互信息，系数 4 对应每次采样对 4 个缓存行各进行一次独立观测，以互信息与采样频率的乘积从信息泄露速率角度刻画侧信道威胁。

缓解措施评估中，对噪声注入和缓存刷新两种方法进行了系统化评估。基准测试中三种 AES 变体在无缓解措施的情况下成功率均为 100%。缓存刷新使成功率与泄露带宽均降至零，防护彻底。噪声注入方面发现：密钥越长，加密轮数越多， p_{miss} 越低，每阶段有效信息量越少，加之两阶段攻击的 32 字节乘积级联放大效应，对噪声亦越敏感。

5.2 未来工作展望

跨平台验证。框架目前在 x86 上已验证通过，但 ARM 和 RISC-V 架构的缓存层次结构存在差异，T 表的缓存行布局可能需要重新适配。虚拟化环境和容器环境中，Hypervisor 或容器运行时可能对 CLFLUSH 指令和共享内存施加额外限制，可行性亦需重新评估。

缓解措施的优化。噪声注入和缓存刷新各有优劣——缓存刷新防护彻底但性能开销大，噪声注入开销可控但防护不彻底。自适应缓解是一个值得探索的方向：根据运行时检测到的侧信道威胁指标动态调整噪声强度，在安全与性能之间寻求动态平衡。

参考文献

- [1] RISTENPART T, TROMER E, SHACHAM H, et al. Hey, You, Get Off of My Cloud: Detecting, Measuring, and Defending Against Resource-Free Cloud Co-Residents[C]//Proceedings of the 16th ACM Conference on Computer and Communications Security. New York, NY: ACM, 2009: 199-212.
- [2] ZHANG Y, JUELS A, REITER M K, et al. Cross-Tenant Side-Channel Attacks in PaaS Clouds[C]//Proceedings of the ACM SIGSAC Conference on Computer and Communications Security. New York, NY: ACM, 2014: 990-1003.
- [3] 张伟娟, 白璐, 凌雨卿, 等. 缓存侧信道攻击与防御[J]. 计算机研究与发展, 2023, 60(1): 206-222. DOI: 10.7544/issn1000-1239.202110774.
- [4] 王崇, 魏帅, 张帆, 等. 缓存侧信道防御研究综述[J]. 计算机研究与发展, 2021, 58(4): 794-810. DOI: 10.7544/issn1000-1239.2021.20200500.
- [5] National Institute of Standards and Technology. Advanced Encryption Standard (AES)[S]. Gaithersburg, MD: National Institute of Standards, 2001.
- [6] OSVIK D A, SHAMIR A, TROMER E. Cache Attacks and Countermeasures: The Case of AES[C]//Topics in Cryptology – CT-RSA 2006. Berlin: Springer, 2006: 1-20.
- [7] KOCHER P, HORN J, FOGH A, et al. Spectre Attacks: Exploiting Speculative Execution[C]//Proceedings of the IEEE Symposium on Security and Privacy. Piscataway, NJ: IEEE, 2019: 1-19.
- [8] LIPP M, SCHWARZ M, GRUSS D, et al. Meltdown: Reading Kernel Memory from User Space[C]//Proceedings of the 27th USENIX Security Symposium. Berkeley, CA: USENIX Association, 2018: 973-990.
- [9] VAN BULCK J, MINKIN M, WEISSE O, et al. Foreshadow: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution[C]//Proceedings of the 27th USENIX Security Symposium. Berkeley, CA: USENIX Association, 2018: 991-1008.
- [10] Van SCHAIK S, MILBURN A, ÖSTERLUND S, et al. RIDL: Rogue In-Flight Data Load[C]//Proceedings of the IEEE Symposium on Security and Privacy. Piscataway, NJ: IEEE, 2019: 88-105.
- [11] SCHWARZ M, LIPP M, MOGHIMI D, et al. ZombieLoad: Cross-Privilege-Boundary Data Sampling[C]//Proceedings of the ACM SIGSAC Conference on Computer and Communications Security. New York, NY: ACM, 2019: 753-768.
- [12] CANELLA C, VAN BULCK J, SCHWARZ M, et al. A Systematic Evaluation of Transient Execution Attacks and Defenses[C]//Proceedings of the 28th USENIX Security Symposium. Berkeley, CA: USENIX Association, 2019: 249-266.

- [13] KOCHER P C. Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems[C]//Advances in Cryptology – CRYPTO '96. Berlin: Springer, 1996: 104-113.
- [14] KOCHER P, JAFFE J, JUN B. Differential Power Analysis[C]//Advances in Cryptology – CRYPTO '99. Berlin: Springer, 1999: 388-397.
- [15] PAGE D. Theoretical Use of Cache Memory as a Cryptanalytic Side-Channel[R]. CSTR-02-003. Bristol: University of Bristol, 2002.
- [16] BERNSTEIN D J. Cache-Timing Attacks on AES[EB/OL]. 2005 [2025-01-15]. <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>.
- [17] PERCIVAL C. Cache Missing for Fun and Profit[C]//Proceedings of BSDCan 2005. Ottawa: BSDCan, 2005.
- [18] TROMER E, OSVIK D A, SHAMIR A. Efficient Cache Attacks on AES, and Countermeasures[J]. Journal of Cryptology, 2010, 23(1): 37-71.
- [19] YAROM Y, FALKNER K. FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack[C]//Proceedings of the 23rd USENIX Security Symposium. Berkeley, CA: USENIX Association, 2014: 719-732.
- [20] BENDER N, van de POL J, SMART N P, et al. "Ooh Aah... Just a Little Bit": A Small Amount of Side Channel Can Go a Long Way[C]//Cryptographic Hardware and Embedded Systems – CHES 2014. Berlin: Springer, 2014: 75-92.
- [21] LIU F, YAROM Y, GE Q, et al. Last-Level Cache Side-Channel Attacks are Practical[C]//Proceedings of the IEEE Symposium on Security and Privacy. Piscataway, NJ: IEEE, 2015: 605-622.
- [22] GRUSS D, SPREITZER R, MANGARD S. Cache Template Attacks: Automating Attacks on Inclusive Last-Level Caches[C]//Proceedings of the 24th USENIX Security Symposium. Berkeley, CA: USENIX Association, 2015: 897-912.
- [23] GRUSS D, MAURICE C, WAGNER K, et al. Flush+Flush: A Fast and Stealthy Cache Attack[C]//Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA). Berlin: Springer, 2017: 279-299.
- [24] SHUSTERMAN A, KANG L, MELTSEY Y, et al. The Spy in the Sandbox: Practical Cache Attacks in JavaScript and Their Implications[C]//Proceedings of the ACM SIGSAC Conference on Computer and Communications Security. New York, NY: ACM, 2019: 1407-1420.
- [25] GRUSS D, MAURICE C, MANGARD S. Rowhammer.js: A Remote Software-Induced Fault Attack in JavaScript[C]//Proceedings of the 13th International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment. Berlin: Springer, 2016: 300-321.
- [26] 吴晓慧, 贺也平, 马恒太, 等. 微架构瞬态执行攻击与防御方法[J]. 软件学报, 2020, 31(2): 544-563.
- [27] 杨帆, 张倩颖, 施智平, 等. 可信执行环境软件侧信道攻击研究综述[J]. 软件学报, 2023, 34(1): 381-403. DOI: 10.13328/j.cnki.jos.006501.
- [28] 唐博文, 武成岗, 王喆. 处理器时间侧信道攻防技术综述[J]. 高技术通讯 (中文), 2024, 34(5): 439-452. DOI: 10.3772/j.issn.1002-0470.2024.05.001.

- [29] GE Q, YAROM Y, COCK D, et al. A Survey of Microarchitectural Timing Attacks and Countermeasures on Contemporary Hardware[J]. *Journal of Cryptographic Engineering*, 2018, 8(1): 1-27.
- [30] ACIICMEZ O, KOÇ C K. Trace-Driven Cache Attacks on AES[EB/OL]. 2006 [2025-01-15]. <https://eprint.iacr.org/2006/138>.
- [31] ACIICMEZ O, KOÇ C K, SEIFERT J P. On the Power of Simple Branch Prediction Analysis[C]//*Proceedings of the 2nd ACM Symposium on Information, Computer and Communications Security (ASIACCS)*. New York, NY: ACM, 2007: 312-320.
- [32] NEVE M, SEIFERT J P. Advances on Access-Driven Cache Attacks on AES[C]//*Selected Areas in Cryptography – SAC 2006*. Berlin: Springer, 2007: 147-162.
- [33] GULLASCH D, BANGERTER E, KRENN S. Cache Games – Bringing Access-Based Cache Attacks on AES to Practice[C]//*Proceedings of the IEEE Symposium on Security and Privacy*. Piscataway, NJ: IEEE, 2011: 490-505.
- [34] IRAZOQUI G, İNCI M S, EISENBARTH T, et al. Wait a Minute! A Fast, Cross-VM Cache Attack on AES[C]//*Research in Attacks, Intrusions and Defenses (RAID)*. Berlin: Springer, 2014: 299-316.
- [35] IRAZOQUI G, EISENBARTH T, SUNAR B. Cross-Processor Cache Attacks[C]//*Proceedings of the 11th ACM on Asia Conference on Computer and Communications Security*. New York, NY: ACM, 2016: 353-364.
- [36] WANG Z, LEE R B. New Cache Designs for Thwarting Software Cache-Based Side Channel Attacks[C]//*Proceedings of the 34th International Symposium on Computer Architecture*. New York, NY: ACM, 2007: 494-505.
- [37] KIM T, PEINADO M, MAINAR-RUIZ G. STEALTHMEM: System-Level Protection Against Cache-Based Side Channel Attacks in the Cloud[C]//*Proceedings of the 21st USENIX Security Symposium*. Berkeley, CA: USENIX Association, 2012: 189-204.
- [38] BRASSER F, MÜLLER U, DMITRIENKO A, et al. Software Grand Exposure: SGX Cache Attacks Are Practical[C]//*Proceedings of the 11th USENIX Workshop on Offensive Technologies (WOOT)*. Vancouver, BC: USENIX Association, 2017.

致 谢

时光荏苒，本科阶段的学习即将画上句号。在完成本篇毕业论文之际，我要向所有给予我帮助和支持的人表达最诚挚的感谢。

谢谢我的导师鲁赵骏老师。从选题、研究方法、实验设计到论文写作，鲁老师每个阶段都给了我很多指导和耐心的帮助。

谢谢智能与计算学部的各位老师。老师们讲授的专业课让我印象深刻，让我对计算机越钻越有兴趣。

谢谢我的同学和朋友。这几年一起上课、一起熬夜、一起成长。有他们在的大学生活充满欢声笑语。

谢谢父母和家人们，你们一直是我坚实的后盾。没有你们的辛勤付出和默默支持，我没办法安心把书读好，顺利走到今天。

谢谢所有参考文献的作者。各位的研究成果给了这篇论文不可或缺的理论基础和方法参考。

毕业不是终点，是另一个起点。我会带着在天津大学学到的知识，继续在学术领域探索前行。